

**TINYML-POWERED REAL-TIME NETWORK
THREAT DETECTION & PREVENTION FOR SMEs**

25-26J-056

Thesis Report

Siriwardhanage Tharusha Dilshan Thilakarathna

(IT22578914)

B.Sc. (Hons) Degree in Information Technology Specialized in
Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology Sri Lanka

July 2025

**TINYML-POWERED REAL-TIME NETWORK
THREAT DETECTION & PREVENTION FOR SMEs**

25-26J-056

Thesis Report

Siriwardhanage Tharusha Dilshan Thilakarathna

(IT22578914)

B.Sc. (Hons) Degree in Information Technology Specialized in
Cyber Security

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology Sri Lanka

July 2025

ABSTRACT

Small and medium-sized enterprises (SMEs) face growing cyber threats, but many lack the budget and technical resources needed for advanced security solutions. This study proposes a TinyML-powered real-time network threat detection and prevention system for SMEs using Raspberry Pi 4 as a lightweight edge device. The aim was to develop a practical and low-cost solution that can detect malicious traffic in real time, apply automated response, and provide monitoring through a web dashboard.

To achieve this, selected records from CICIDS2017 and CSE-CIC-IDS2018 were merged, cleaned, and harmonized into a final six-class dataset: Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance. Random Forest feature importance was used to reduce the feature space to the top 20 features. Several machine learning models were evaluated, and the MLP was selected as the final model because it offered the best balance between accuracy and lightweight deployment suitability.

The final model was fine-tuned and deployed in TensorFlow Lite float32 format on Raspberry Pi 4. The runtime pipeline included live packet capture, flow conversion, real-time inference, temporary IP blocking using nftables, and dashboard-based monitoring. The deployed system achieved 98.87% accuracy, with an average pure inference time of 0.0202 ms and preprocessing plus inference time of 3.8578 ms.

Overall, the study demonstrates that a Raspberry Pi-based TinyML approach can provide an effective and practical real-time network threat detection and prevention solution for SME environments.

Keywords: TinyML, network intrusion detection and prevention, Raspberry Pi 4, real-time threat detection, nftables

ACKNOWLEDGEMENT

First and foremost, I would like to express my sincere gratitude to my supervisor, Ms. Deemantha Siriwardhana, and co-supervisor, Ms. Ayesha Wijesooriya, of the Department of Computer Systems Engineering at the Sri Lanka Institute of Information Technology, for their valuable guidance, continuous support, and constructive feedback throughout this research. Their advice and encouragement greatly contributed to the successful completion of this work.

I would also like to extend my heartfelt appreciation to my project teammates, Dulanji Himasha, Rajani Piyarathna, and Chalana Ranwala, for their cooperation, shared effort, and teamwork throughout the development of the UNetrix research project. The successful integration of our individual components into a unified system was made possible through collaboration and mutual support.

Finally, I would like to thank my family for their constant encouragement, patience, and unwavering support throughout my degree programme and during the completion of this research.

TABLE OF CONTENTS

Abstract	iii
Acknowledgement.....	iv
List of Figures	vii
List of Tables.....	vii
List Of Abbreviations	viii
1. Introduction	1
1.1 Problem Statement	2
1.2 Research Gap	3
1.3 Aim of the Study	4
1.4 Research Objectives	4
1.5 Scope of the Study	5
1.6 Research Contributions	6
2. Literature Review.....	8
2.1 Network Intrusion Detection and Prevention Systems.....	8
2.2 Machine Learning and TinyML for Intrusion Detection.....	10
2.3 Raspberry Pi and Edge-Based Security Systems	11
2.4 Public Intrusion Detection Datasets	12
2.5 Dataset Quality and Labeling Issues	13
2.6 Related Work Comparison	15
2.7 Identified Research Gap	17
3. Methodology	19
3.1 Overview of the Proposed System	19
3.2 Dataset Preparation and Label Harmonization.....	20
3.3 Feature Selection and Model Development	21
3.4 Proposed System Architecture	23
3.5 Inline Deployment and Runtime Workflow	24
3.6 Detection, Prevention, and Dashboard Integration	25
4. Implementation	26
4.1 Hardware Environment	26
4.2 Software Environment.....	27

4.3 Real-Time Packet Capture and Flow Conversion	28
4.4 Inference Module Implementation	29
4.5 Prevention Module Implementation	30
4.6 Dashboard Implementation	31
5. Results and Discussion.....	32
5.1 Dataset Results	32
5.2 Feature Selection Results	33
5.3 Model Comparison Results	34
5.4 Final MLP Performance Evaluation	36
5.5 Runtime Performance on Raspberry Pi	38
5.6 Resource Usage Analysis	39
5.7 Prevention Validation Using nftables	41
5.8 Dashboard Validation	42
5.9 Discussion	44
5.10 Limitations	45
6. Commercialization and Practical Deployment.....	46
6.1 SME Need and Practical Relevance	47
6.2 Cost and Deployment Considerations	48
6.3 Potential Productization	49
6.4 Future Scalability and Real-World Use.....	50
7. Conclusion and Future Work.....	52
7.1 Conclusion.....	52
7.2 Future Work.....	53
References	55
Appendices.....	57
Appendix A: Dataset File Selection and Label Harmonization Details	57
Appendix B: Full Feature List and Selected Features	60
Appendix C: Final MLP Architecture and Hyperparameter Settings.....	63

LIST OF FIGURES

Figure 3.1: Dataset Preparation Flowchart.....	20
Figure 3.2: System Architecture Diagram.....	23
Figure 5.1: Final Class Distribution of the Merged Dataset.....	32
Figure 5.2: Top 20 Selected Features Based on Random Forest Importance.....	34
Figure 5.3: Confusion Matrix of MLP Model	37
Figure 5.4: nftables Block Lifecycle	41
Figure 5.5: NIDS Web Dashboard.....	43

LIST OF TABLES

Table 2.1 Comparison of related work	15
Table 5.1: Model Comparison Results	35
Table 5.2 Per-Class Performance of the MLP Model.....	36
Table 5.3 Runtime Performance on Raspberry Pi	38
Table 5.4: Resource Usage Analysis	40

LIST OF ABBREVIATIONS

CPU – Central Processing Unit

CSV – Comma-Separated Values

DDoS – Distributed Denial of Service

DoS – Denial of Service

IDS – Intrusion Detection System

IDPS – Intrusion Detection and Prevention System

IoT – Internet of Things

IP – Internet Protocol

IPS – Intrusion Prevention System

LAN – Local Area Network

ML – Machine Learning

MLP – Multilayer Perceptron

NIDS – Network Intrusion Detection System

NIDPS – Network Intrusion Detection and Prevention System

RAM – Random Access Memory

RF – Random Forest

SME – Small and Medium-sized Enterprise

TFLite – TensorFlow Lite

TinyML – Tiny Machine Learning

XAI – Explainable Artificial Intelligence

1. INTRODUCTION

Small and medium-sized enterprises (SMEs) are becoming increasingly dependent on computer networks, internet services, and digital platforms for their daily operations. As a result, they are also becoming more exposed to cyber threats such as denial-of-service attacks, botnet activity, brute-force attempts, and reconnaissance-based attacks. Unlike large organizations, SMEs often do not have enough budget, infrastructure, or dedicated security teams to deploy and manage expensive enterprise-level security solutions. This makes them more vulnerable to cyber incidents and increases the need for affordable and practical security mechanisms that can still provide effective protection [1].

One important approach used in network security is the intrusion detection and prevention system (IDPS). These systems are designed to monitor network traffic, identify malicious activity, and in some cases respond automatically. In recent years, machine learning-based intrusion detection has received a lot of attention because it can identify abnormal traffic patterns beyond fixed rule-based signatures. At the same time, research has also shown that intrusion detection is moving toward lightweight and edge-based solutions, especially for IoT and small-network environments where hardware resources are limited [2], [3], [4].

Raspberry Pi has become a popular platform in this area because it offers a low-cost and practical option for edge deployment. Previous studies have shown that Raspberry Pi can be used to support lightweight security functions such as traffic monitoring and intrusion detection in constrained environments [3], [4], [5]. However, many earlier studies mainly focus on detection or alerting, while fewer systems combine real-time multiclass detection, automated prevention, and dashboard-based monitoring in one integrated platform. This is especially important for SMEs, where a useful security solution should not only detect threats, but also provide a simple way to respond to them and monitor what is happening in the network [3]–[5].

Based on this background, this study focuses on developing a TinyML-powered real-time network threat detection and prevention system for SMEs. The proposed system is designed to run on a Raspberry Pi 4 as an inline bridge, detect multiple categories of malicious traffic using a lightweight machine learning model, apply temporary blocking using nftables, and display events through a web dashboard. The goal is to provide a practical and affordable edge-based security solution that can support SME environments more effectively.

1.1 Problem Statement

SMEs face increasing cyber risks, but many of them do not have access to advanced security infrastructure or the resources needed to maintain complex network defense solutions [1]. Traditional enterprise-grade intrusion detection and prevention systems can be costly, difficult to manage, or too resource-intensive for small organizations. As a result, many SMEs remain exposed to common network threats without having a practical system that can detect malicious activity in real time and respond automatically.

Although many previous studies have proposed machine learning-based intrusion detection methods, several limitations still remain. First, a large number of studies focus mainly on offline classification accuracy using benchmark datasets, rather than on a full deployed system that works on live traffic [2], [3]. Second, some lightweight or Raspberry Pi-based systems mainly provide detection and alerting, but do not clearly implement practical automated prevention [3], [4], [5]. Third, highly accurate models are not always suitable for deployment on constrained devices, since they may be too large or too computationally expensive for real-time edge use [3], [4], [6].

Another challenge is that public benchmark datasets such as CICIDS2017 and CSE-CIC-IDS2018 contain issues such as label inconsistencies, duplicated data, and feature-generation problems, which can affect the reliability of machine learning results if they are used without proper cleaning and harmonization [2], [7]. Therefore, there is still a need for a practical intrusion detection and prevention system that is not only accurate, but also lightweight,

deployable, and supported by a cleaned multiclass dataset and a usable monitoring interface.

Based on these issues, the main problem addressed in this study is the lack of a low-cost, lightweight, and practical real-time network threat detection and prevention solution for SMEs that can run on an edge device, detect multiple attack categories, trigger automated temporary blocking, and provide clear monitoring support through a dashboard.

1.2 Research Gap

The literature shows that there has been significant progress in machine learning-based intrusion detection, especially for IoT, edge, and lightweight security environments. Many studies have proposed compact IDS models, reduced feature sets, and low-cost deployment platforms such as Raspberry Pi. Other works have also shown that public datasets like CICIDS2017 and CSE-CIC-IDS2018 are widely used for building and testing intrusion detection models [2], [3], [4], [8].

However, several important gaps still remain. First, many studies focus mainly on offline model performance rather than on a complete real-time deployed system. Second, some Raspberry Pi-based or lightweight IDS solutions mainly support detection and alerting, but do not clearly implement an effective automated prevention mechanism. Third, even when prevention is discussed, fewer studies combine multiclass attack detection, inline Raspberry Pi deployment, confidence-based blocking, and dashboard-based monitoring in one integrated solution [3], [4], [5], [9].

Another gap is related to the practical reliability of dataset-driven IDS development. Although public benchmark datasets are useful, earlier studies have reported label inconsistencies, duplicated records, and feature-generation issues in widely used intrusion detection datasets. This means that strong classification accuracy alone is not enough unless the dataset has also been carefully cleaned and harmonized [2], [7]. In addition, highly accurate models

are not always suitable for edge deployment because they may be too large or computationally expensive for devices such as Raspberry Pi [3], [4], [6].

Therefore, the main research gap identified in this study is the lack of a practical SME-oriented edge security system that combines a cleaned multiclass dataset, a lightweight deployable model, real-time inline traffic analysis, automated temporary IP blocking, and dashboard-based monitoring in a single platform. This study addresses that gap by proposing a TinyML-powered real-time network threat detection and prevention system for SMEs using Raspberry Pi 4 as the edge deployment device.

1.3 Aim of the Study

The aim of this study is to develop a TinyML-powered real-time network threat detection and prevention system for SMEs that can operate on a Raspberry Pi 4 as a low-cost edge device. The proposed system is intended to detect multiple categories of malicious network traffic using a lightweight machine learning model, apply temporary automated prevention through nftables, and provide real-time monitoring through a web dashboard. The study focuses on creating a practical solution that balances detection accuracy, lightweight deployment, automated response, and usability in an SME network environment [1], [3], [4].

1.4 Research Objectives

The main objective of this study is to develop a TinyML-powered real-time network threat detection and prevention system for SMEs using Raspberry Pi 4 as a lightweight edge device.

The specific objectives of the study are as follows:

- 1 To prepare a clean and harmonized multiclass intrusion detection dataset using selected records from CICIDS2017 and CSE-CIC-IDS2018.
- 2 To identify the most relevant flow-based features and reduce the input space to a smaller feature set suitable for lightweight model deployment.

- 3 To train and evaluate different machine learning models for multiclass network threat detection and select the most suitable model based on both performance and deployment efficiency.
- 4 To deploy the selected model on Raspberry Pi 4 using TensorFlow Lite for real-time inference.
- 5 To implement a confidence-based prevention mechanism using nftables to temporarily block malicious source IP addresses.
- 6 To develop a web dashboard to display detected attacks, blocked IPs, alerts, and monitoring information in real time.
- 7 To evaluate the final system in terms of classification performance, runtime efficiency, resource usage, prevention behavior, and practical suitability for SME environments.

1.5 Scope of the Study

This study focuses on the design and development of a lightweight real-time network threat detection and prevention system for SMEs using Raspberry Pi 4 as an edge device. The system is limited to network-based monitoring, where traffic passing through the Raspberry Pi inline bridge is captured, converted into flow-based features, and analyzed using a deployed machine learning model. The final model is designed to classify traffic into six classes: Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance.

The scope of the study includes dataset preparation, label harmonization, feature selection, model training, TensorFlow Lite deployment, runtime inference, automated temporary IP blocking using nftables, and dashboard-based monitoring. It also includes evaluation of the final system in terms of classification performance, inference time, resource usage, prevention validation, and practical deployment on Raspberry Pi 4.

However, this study does not cover every possible type of network attack. Attack categories such as Infiltration, Web Attack, and Heartbleed were excluded from the final system due to dataset quality issues or low sample counts. The study also does not aim to build a large-scale enterprise security

platform, cloud-based distributed IDS, or a full commercial product. Instead, it focuses on a practical prototype that demonstrates how a low-cost edge device can provide real-time detection, prevention, and monitoring for SME environments.

In addition, the deployment and testing were carried out in a controlled experimental environment. Therefore, while the system demonstrates strong potential for real-world SME use, the study does not fully cover long-term production deployment across highly diverse network conditions. Future work can expand the scope by supporting more attack classes, improving preprocessing efficiency, and testing the system in broader live environments.

1.6 Research Contributions

This study makes several contributions to the development of lightweight network security solutions for SME environments.

First, it proposes a TinyML-powered real-time network threat detection and prevention system that is specifically designed for small and medium-sized enterprises. Unlike many previous studies that mainly focus on offline model evaluation, this work combines dataset preparation, model development, edge deployment, automated prevention, and monitoring into a single practical system.

Second, the study develops a cleaned and harmonized multiclass dataset using selected records from CICIDS2017 and CSE-CIC-IDS2018. During this process, inconsistent labels were merged into common attack categories, and unreliable or low-sample classes were removed. This created a more stable dataset for the final six-class classification problem.

Third, the study applies feature selection to reduce the input space and make the final model more suitable for edge deployment. By selecting the most important flow-based features, the system reduces processing complexity while maintaining strong classification performance.

Fourth, the study evaluates several machine learning models and selects a lightweight MLP model based on both accuracy and deployment practicality. The final model is then fine-tuned and deployed in TensorFlow Lite format on Raspberry Pi 4, demonstrating that real-time multiclass intrusion detection can be achieved on a low-cost edge device.

Fifth, the study implements a confidence-based prevention mechanism using nftables, allowing the system to temporarily block malicious source IP addresses for a fixed duration. This makes the final solution more practical than a detection-only IDS.

Finally, the study includes a web dashboard for real-time monitoring. The dashboard displays detected attacks, blocked IPs, confidence scores, and alert information, improving the usability of the system in an SME environment.

Overall, the main contribution of this study is the development of an integrated lightweight edge-based security solution that combines real-time detection, automated prevention, and monitoring in a single Raspberry Pi-based platform.

2. LITERATURE REVIEW

This chapter reviews the main concepts and previous studies related to the proposed system. The discussion focuses on network intrusion detection and prevention systems, machine learning and lightweight intrusion detection approaches, Raspberry Pi and edge-based security deployment, and the datasets commonly used in this research area. The purpose of this chapter is to build the background for the study and to show how the proposed system fits into existing research on practical and lightweight network security solutions [2], [3], [4].

In recent years, intrusion detection research has moved beyond traditional server-based and alert-only systems toward solutions that are more lightweight, edge-aware, and suitable for constrained environments. This change is especially important in small and medium-sized enterprises, where security solutions need to be affordable, easy to deploy, and capable of working with limited hardware resources. At the same time, earlier studies have shown that the reliability of intrusion detection systems also depends heavily on the datasets, features, and tools used during development [2], [3], [10], [1].

Based on this background, the following sections examine the main ideas, technologies, and research findings that are directly related to the proposed TinyML-powered real-time network threat detection and prevention system. The chapter also identifies the limitations of current approaches and uses them to justify the need for a lightweight and practical solution for SME environments [3], [4], [1].

2.1 Network Intrusion Detection and Prevention Systems

A network intrusion detection and prevention system is used to monitor network activity, identify suspicious or malicious behavior, and in some cases respond automatically. According to NIST, intrusion detection and prevention systems are designed to identify possible security incidents, record information about them, and report them, while intrusion prevention systems can also attempt to stop the detected activity. NIST also explains that these systems can be grouped into several categories, including network-based, host-based, wireless, and

network behavior analysis systems. Since the proposed project monitors traffic passing through a network bridge, it mainly belongs to the network-based intrusion detection and prevention category [11].

Traditional intrusion detection systems often relied on signature-based detection, where known patterns of attacks were matched against observed traffic. This approach is useful for identifying previously known threats, but it has limitations when dealing with new or modified attacks. Because of this, many modern systems use anomaly-based or machine learning-based methods, which teach traffic behavior from data and try to separate benign activity from malicious behavior. Recent reviews and lightweight IDS studies show that machine learning has become a major research direction, especially in IoT and edge environments where rule-based approaches alone may not be sufficient [2], [3], [4], [12].

Another important issue is the difference between detection-only systems and systems that support automated prevention. Many lightweight and Raspberry Pi-based studies focus mainly on detecting attacks and generating alerts, but fewer systems include a practical response mechanism such as temporary blocking or policy enforcement. For SME environments, this difference is important because a useful security solution should provide not only visibility, but also at least a basic automatic response to high-confidence threats. This makes intrusion prevention an important extension of intrusion detection in practical edge deployments [3], [4], [1], [5].

Overall, network intrusion detection and prevention systems remain an important part of modern cyber defense because they provide traffic monitoring, attack identification, and in some cases immediate response. Their importance becomes even greater in SME and edge environments, where systems must be lightweight, low-cost, and still effective enough to handle real threats. These characteristics directly connect to the aims of the proposed system, which combines network-based monitoring, machine learning-based classification, automated prevention, and real-time dashboard visibility on a Raspberry Pi platform. [2]– [5]

2.2 Machine Learning and TinyML for Intrusion Detection

Machine learning has become an important approach in intrusion detection because it can learn traffic patterns from data and identify malicious behavior beyond fixed signatures. This is especially useful in modern networks, where attack behavior can change and new threats may not match previously known rules. Recent review studies and lightweight IDS research show that machine learning-based models are now widely used in network intrusion detection, particularly in IoT and edge environments where traffic behavior is dynamic and traditional rule-only approaches are often not enough [2]– [4].

However, using machine learning in constrained environments creates another challenge. Many high-performing models can be too large or too computationally expensive for direct deployment on edge devices. For this reason, recent research has increasingly focused on lightweight intrusion detection, where the goal is to maintain strong detection performance while reducing model size, computational cost, and latency. Studies such as Realguard and other lightweight IDS approaches show that compact models, reduced feature sets, and efficient deployment strategies are necessary when the target platform is an IoT gateway or a Raspberry Pi-like device [3], [4]. More recent work has continued in the same direction by proposing lightweight two-stage intrusion detection methods for IoT and data-condensation plus feature-stability approaches to reduce computation while preserving accuracy [10], [13].

TinyML is closely related to this idea because it focuses on deploying machine learning models on resource-constrained devices. In intrusion detection, TinyML does not simply mean using a small model; it also involves selecting useful features, reducing runtime overhead, and choosing a deployment format that can run efficiently on limited hardware. Previous studies have explored model compression, quantization, and lightweight neural approaches for this purpose, showing that smaller models can still provide strong detection performance when they are carefully designed [14], [15]. In addition, recent research on explainable feature-selection methods has shown that reducing the

number of features can improve efficiency while still preserving important security-related information, which is highly relevant for practical edge deployment [12].

For this reason, the proposed study follows the same general research direction by selecting a compact feature set, evaluating several models, and finally deploying a lightweight model on Raspberry Pi 4 using TensorFlow Lite. Rather than selecting the highest-accuracy model only, the focus is on achieving a balance between classification performance, model size, runtime efficiency, and deployment practicality. This makes machine learning and TinyML directly relevant to the design of the proposed real-time network threat detection and prevention system for SMEs [3], [4], [14], [15].

2.3 Raspberry Pi and Edge-Based Security Systems

Raspberry Pi has become a popular platform in lightweight security research because it is low-cost, compact, and energy-efficient, while still providing enough computing power for practical edge-based applications. These characteristics make it attractive for small-scale deployments where expensive dedicated security hardware may not be practical. In intrusion detection research, Raspberry Pi is often used as a gateway or edge device that can monitor traffic close to the source and perform local security tasks without depending fully on cloud infrastructure [3], [5], [9].

Several studies have shown that Raspberry Pi can support network intrusion detection in constrained environments. For example, Realguard presented a lightweight network intrusion detection system for IoT gateways, showing that gateway-based security can be implemented with low computational overhead while still detecting multiple attacks in real time [3]. Similarly, PITI demonstrated that Raspberry Pi can be used to protect IoT environments through intrusion detection, but its main focus was on detection and alerting rather than practical automated prevention [5]. Other Raspberry Pi-based studies have also explored firewall and IDS combinations, showing that the platform can host multiple security functions within a small and affordable system [16].

At the same time, earlier work also shows that Raspberry Pi-based security systems face practical limitations. The exploratory study on intelligent cyber defense systems for SMEs and smart homes points out that many previous studies mainly focused on whether Raspberry Pi can technically run IDS software, while giving less attention to long-term deployment, mixed traffic environments, and practical SME usage [9]. This is important because a real-world SME solution must not only detect attacks, but also remain lightweight, manageable, and useful in day-to-day operation. Therefore, Raspberry Pi is not just a low-cost substitute for larger hardware, but a platform that requires careful design choices in terms of model size, feature processing, runtime efficiency, and response mechanisms [3], [9].

Based on these findings, Raspberry Pi is a suitable platform for the proposed study because it supports the main goals of edge deployment, low-cost implementation, and practical SME-oriented security monitoring. However, the literature also shows that simply running an IDS on Raspberry Pi is not enough. What matters is how effectively the device can combine lightweight detection, practical prevention, and usable monitoring support in a real deployment environment. This is the gap that the proposed system aims to address [3], [5], [9].

2.4 Public Intrusion Detection Datasets

Public intrusion detection datasets play an important role in machine learning-based IDS research because they provide labeled traffic records for training and evaluation. Without such datasets, it is difficult to compare models fairly or reproduce experimental results. For this reason, many recent intrusion detection studies rely on benchmark datasets that contain both benign traffic and different types of attack behavior [2], [8].

Among the available benchmark datasets, CICIDS2017 and CSE-CIC-IDS2018 are two of the most widely used in recent research. CICIDS2017 was created to represent realistic network behavior by combining normal user activity with multiple attack scenarios such as DoS, DDoS, brute force, botnet activity,

infiltration, and port scanning [8]. CSE-CIC-IDS2018 was later introduced as an extended benchmark with a wider collection of attack scenarios and a larger volume of traffic, which made it useful for more recent machine learning and deep learning studies [2], [12]. These datasets are particularly relevant to this study because they contain the main attack categories used in the final six-class problem.

Another important reason for using these datasets is that they are flow-based, which means the traffic is represented using statistical flow features rather than raw packets. This makes them suitable for building lightweight IDS models, since flow features are easier to process than full packet streams in many edge deployment scenarios. Previous studies have also used these datasets in combination with CICFlowMeter-style feature generation, which matches the general approach used in the proposed system [2], [8].

However, even though these datasets are popular, they are not perfect. Recent research has shown that CICIDS2017 and CSE-CIC-IDS2018 contain some issues such as label inconsistencies, duplicated records, and feature-generation problems [2], [12]. Because of this, they are useful as benchmark datasets, but they often need cleaning and harmonization before they can be used reliably for model development. This is one of the reasons why dataset preparation became an important part of the proposed study.

2.5 Dataset Quality and Labeling Issues

Although public intrusion detection datasets are useful for training and testing machine learning models, their quality has a strong influence on the reliability of the final results. If a dataset contains incorrect labels, duplicated records, inconsistent attack names, or unstable feature values, the model may learn misleading patterns instead of meaningful traffic behavior. Because of this, dataset quality has become an important concern in modern intrusion detection research, especially when benchmark datasets are used to compare different models [2], [12].

This issue is especially relevant for CICIDS2017 and CSE-CIC-IDS2018, since both datasets are widely used in machine learning-based intrusion detection studies. A recent review on intrusion detection datasets pointed out that these datasets are popular because of their realistic traffic structure and flow-based features, but later studies also identified several weaknesses such as label inconsistencies, class imbalance, duplicated data, and problems related to feature generation [2]. In addition, the error-analysis study on these datasets reported previously undocumented issues in attack orchestration, feature generation, dataset documentation, and labeling. These findings show that strong model performance on benchmark datasets does not always mean that the dataset itself is fully reliable [2], [12].

Labeling problems are particularly important in multiclass intrusion detection. If similar attack types are labeled differently across datasets, or if some records are incorrectly assigned to a class, the final classification problem becomes less stable. This can affect both training and evaluation, and it may also lead to unrealistic conclusions about model performance. For this reason, several researchers have emphasized the need for cleaning, relabeling, and harmonization before using public datasets in practical IDS development [2], [12].

These issues directly influenced the design of the proposed study. Instead of using the raw labels from CICIDS2017 and CSE-CIC-IDS2018 without modification, the datasets were carefully cleaned and harmonized before training. Unreliable or less suitable classes were removed, and related attack labels were merged into a smaller number of final target categories. This helped create a more consistent dataset for model development and improved the relevance of the final classification problem to the intended SME deployment scenario. Therefore, dataset quality and labeling issues were not treated as minor preprocessing steps, but as an important part of building a reliable lightweight intrusion detection and prevention system [2], [12].

2.6 Related Work Comparison

Several previous studies have explored lightweight intrusion detection in IoT, edge, and Raspberry Pi-based environments. However, most of them focus on only part of the full problem, such as lightweight detection, dataset-based classification, or low-cost deployment. A comparison of the most relevant studies shows that while many systems achieve good detection performance, fewer of them combine real-time detection, automated prevention, and user-facing monitoring in one practical edge device [3], [8], [5], [6].

Study	Platform	Prevention	Dashboard	Main Limitation
Realguard [3]	IoT gateway / edge	Limited	Limited	Focuses mainly on lightweight detection
Soe et al. [4]	IoT environment	No	No	Focuses on efficient detection only
PITI [5]	Raspberry Pi / IoT	Limited	Basic	Prevention is not a major focus
Tirumala et al. [9]	SMEs / smart homes	Not fully shown	Not fully shown	More exploratory than fully implemented
Mogos [16]	Raspberry Pi	Yes	No	Not focused on ML-based multiclass detection
Alwaisi et al. [6]	Constrained IoT	Yes	Limited	Not focused on Raspberry Pi inline deployment
Two-stage lightweight IDS [10]	IoT environment	No	No	Focuses on model efficiency only
Proposed Study	Raspberry Pi 4 inline bridge	Yes	Yes	Integrates detection, prevention, and monitoring

Table 2.1 Comparison of related work

For example, Realguard proposed a lightweight intrusion detection system for IoT gateways and showed that low-overhead edge detection is possible in practice [3]. PITI also demonstrated Raspberry Pi-based intrusion detection for IoT protection, but its main contribution was centered on alerting and monitoring rather than a complete automated prevention workflow [5]. Similarly, other Raspberry Pi-based studies showed that the platform can host

security functions such as firewalling and intrusion detection, but they did not always focus on a complete end-to-end multiclass detection and response pipeline for SME environments [16], [9].

Other studies have focused more on the machine learning side of intrusion detection. Research on lightweight IDS models for IoT environments has shown that reducing feature space and model complexity can still provide strong classification performance [4], [6]. Some more recent work has also explored anomaly detection and prevention in constrained IoT systems using lightweight machine learning approaches [6]. These studies are important because they support the idea that a small and efficient model can still be useful in a real deployment setting. However, many of them are mainly focused on the model itself rather than on a full deployed system with traffic capture, edge inference, IP blocking, and dashboard monitoring [4], [6].

Another issue seen in previous work is that some systems are evaluated mainly in offline conditions using public datasets, while fewer papers show how the solution behaves when deployed on a live edge device with real packet capture and runtime response. In addition, although some papers discuss prevention, practical implementations of temporary automated blocking on a Raspberry Pi-based network defense system are less commonly demonstrated together with a real-time dashboard interface [3], [5], [9].

Therefore, the main gap identified from the literature is not simply the lack of lightweight IDS models, but the lack of an integrated SME-oriented edge solution that combines:

- multiclass attack detection,
- real-time inline deployment on Raspberry Pi,
- confidence-aware automated prevention, and
- dashboard-based operational monitoring.

The proposed study addresses this gap by combining all of these elements into one deployable system designed for practical SME use.

2.7 Identified Research Gap

The reviewed literature shows that there has been significant progress in intrusion detection using machine learning, especially in IoT and edge-based environments. Many studies have proposed lightweight detection models, reduced feature sets, and low-cost platforms such as Raspberry Pi for security monitoring. Other works have also shown that public datasets such as CICIDS2017 and CSE-CIC-IDS2018 can support the development of machine learning-based intrusion detection systems [2], [3], [4], [8].

However, literature also reveals several gaps. First, many studies focus mainly on offline classification performance rather than on a complete real-time deployed system. Second, a number of Raspberry Pi-based or lightweight IDS studies emphasize detection and alerting, but do not clearly implement a practical automated prevention mechanism. Third, although several works discuss edge-based detection, fewer studies combine multiclass attack detection, real-time inline deployment, automatic temporary blocking, and dashboard-based monitoring in a single integrated solution [3], [4], [5], [9].

Another important gap is related to dataset reliability and practical model design. Previous research has shown that popular benchmark datasets may contain labeling inconsistencies, duplicated records, and feature-generation issues, which means they often require careful cleaning and harmonization before being used in model development [2], [12]. In addition, highly accurate models are not always suitable for edge deployment, because they may be too large or computationally expensive for devices such as Raspberry Pi. This creates a need for solutions that focus not only on classification accuracy, but also on lightweight deployment, runtime efficiency, and practical usability [3], [4], [6].

Based on these gaps, this study proposes a TinyML-powered real-time network threat detection and prevention system for SMEs. The proposed work differs from much of the earlier literature by combining:

1. a cleaned and harmonized multiclass dataset,

2. lightweight model deployment on Raspberry Pi 4,
3. confidence-based temporary IP blocking using nftables, and
4. a web dashboard for real-time monitoring.

Therefore, the identified research gap is not simply the absence of lightweight IDS models, but the lack of a practical SME-oriented edge security system that integrates detection, prevention, and monitoring in one deployable platform.

3. METHODOLOGY

This chapter presents a detailed description of the design, development, and implementation of the proposed TinyML-powered real-time network threat detection and prevention system. It covers the overall system architecture, data collection and preprocessing, feature engineering, model development, deployment on resource-constrained hardware, real-time inference pipeline, and the automated prevention mechanism.

3.1 Overview of the Proposed System

The proposed system is a TinyML-powered real-time network threat detection and prevention solution designed for small and medium-sized enterprises (SMEs). It was developed to provide a practical and low-cost security mechanism that can detect malicious traffic and take automated action without requiring expensive hardware. This is important because SMEs are often exposed to cyber threats but usually have limited security resources compared to larger organizations [1].

In this study, the system is deployed on a Raspberry Pi 4 which acts as an inline bridge between the router and the internal SME network. As traffic passes through the device, packets are captured and converted into flow-based features. These features are then processed and given to the deployed TensorFlow Lite model, which classifies the traffic into one of the final target classes: Benign, DoS, DDoS, Brute Force, Botnet, or Reconnaissance. Similar lightweight edge-based IDS approaches have also been explored in previous studies for constrained environments such as IoT gateways and Raspberry Pi-based deployments [3], [4].

After the prediction is made, the system checks the confidence score of the detected class. If the traffic is identified as malicious with high confidence, the source IP is temporarily blocked using nftables. At the same time, the event is recorded in a web dashboard, which displays detected attacks, blocked IP addresses, confidence levels, and alerts. Therefore, the proposed system does

not work only as a detection model, but as a complete lightweight detection and prevention system for SME networks [3], [5].

3.2 Dataset Preparation and Label Harmonization

The dataset used in this study was created by merging selected files from CICIDS2017 and CSE-CIC-IDS2018. These two datasets were selected because they contain both benign traffic and several modern network attack categories, which makes them useful for machine learning-based intrusion detection research [8]. However, the two datasets were not used directly in their original form because their labels, attack names, and data quality were not fully consistent.

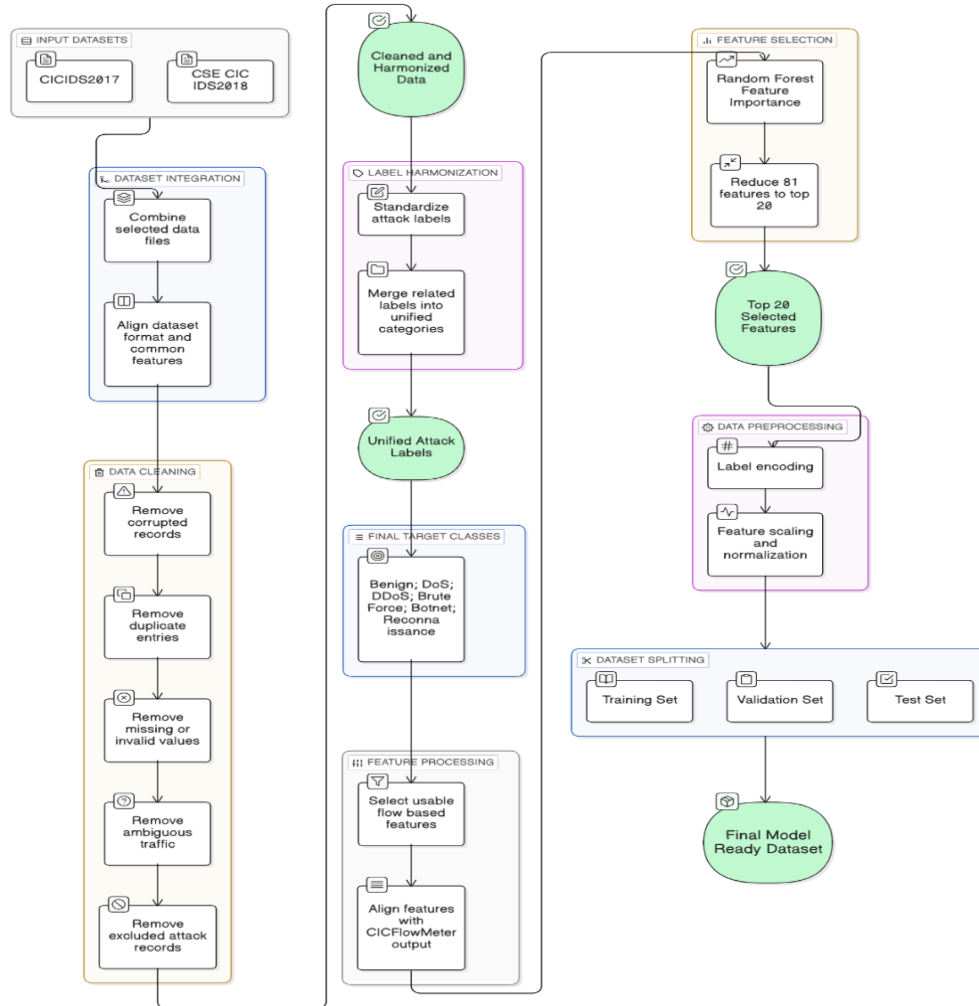


Figure 3.1: Dataset Preparation Flowchart

After selecting the required CSV files, the data from both datasets was combined into a single dataset. During this stage, the feature structure was checked to make sure that the records could be aligned into one common format. A cleaning process was then applied to remove invalid records, noisy entries, and classes that were not suitable for the final model. This step was important because previous studies have shown that CICIDS2017 and CSE-CIC-IDS2018 contain labeling problems, duplicated records, and other dataset quality issues that can affect model reliability if they are not handled properly [7].

Next, a label harmonization process was carried out. Similar attack labels from both datasets were grouped into common final categories. All DoS-related labels were merged into DoS, and all DDoS-related labels were merged into DDoS. FTP- and SSH-based brute-force labels were merged into Brute Force, Bot was renamed as Botnet, and Port Scan was mapped to Reconnaissance. This step was necessary to build a simpler and more practical multiclass detection problem that matched the final system objectives.

Some attack classes were excluded from the final dataset. Infiltration was removed because earlier work reported corruption and reliability issues related to this class [7]. Web Attack and Heartbleed were also removed because they had relatively low sample counts compared to the selected main attack classes. After preprocessing and harmonization, the final dataset contained six target classes: Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance. This final dataset was then used for feature selection, model training, fine-tuning, and deployment.

3.3 Feature Selection and Model Development

Once the final dataset was prepared, feature selection and model development were carried out with the goal of building a classifier that was both accurate and suitable for deployment on a resource-constrained edge device. In intrusion detection research, feature reduction is commonly used to decrease computational cost, simplify preprocessing, and improve the practicality of deployment on devices such as Raspberry Pi without causing a major loss in

detection performance [3], [4], [8]. The merged dataset used in this study originally contained 81 columns, including label-related fields. Before model training, only predictive traffic features were retained, and the remaining flow-based attributes were ranked using Random Forest feature importance. This approach is consistent with earlier intrusion detection studies that evaluated feature usefulness using CICFlowMeter-style traffic attributes and showed that a smaller feature subset can still preserve strong detection capability [4], [8].

Based on the feature-ranking stage, the input space was reduced to the top 20 features: **Init Fwd Win Bytes, Fwd Seg Size Min, Fwd Header Length, Init Bwd Win Bytes, Bwd Packet Length Mean, Bwd Packet Length Std, Subflow Fwd Bytes, Fwd Packet Length Max, Fwd Packets Length Total, Bwd Header Length, Avg Bwd Segment Size, Fwd Packet Length Mean, Bwd Packets/s, Subflow Bwd Bytes, Avg Fwd Segment Size, Packet Length Std, Fwd IAT Total, Packet Length Max, Total Fwd Packets, and Avg Packet Size**. These features mainly represent packet length behavior, header information, flow timing, transmission rate, and subflow statistics. By reducing the original feature space to these 20 selected features, the preprocessing and inference pipeline became simpler and more suitable for real-time edge execution.

After feature selection, several baseline machine learning models were evaluated, including Logistic Regression, Random Forest, XGBoost, and Multilayer Perceptron (MLP). Although Random Forest and XGBoost produced slightly higher offline classification performance, the final model was not selected based on accuracy alone. Since the proposed system had to run on a Raspberry Pi 4 in real time, model compactness and deployment efficiency were also treated as important design criteria, which is also emphasized in previous lightweight IDS studies for constrained environments [3], [4], [14]. Based on hyperparameter tuning, the final MLP was configured as a **6-class multiclass classifier** with **20 input features** and **6 output neurons**. The selected architecture consisted of **three hidden layers** with **32, 32, and 64 neurons**, respectively, and a **learning rate of 0.001**.

The final tuned MLP achieved **98.87% accuracy** and was selected as the deployment model because it offered the best balance between strong detection performance and lightweight deployment suitability. The saved .keras model size was **79.28 KB**, and after conversion to **TensorFlow Lite float32** format, the deployed model size was reduced to **18.96 KB**. This compact size made the model significantly more suitable for real-time Raspberry Pi deployment than larger alternatives such as Random Forest and XGBoost. Therefore, the final model was selected not only because of its multiclass detection performance, but also because it supported the lightweight, edge-based design goals of the proposed system.

3.4 Proposed System Architecture

The proposed system architecture was designed to provide a lightweight and practical network threat detection and prevention solution for SME environments. The system is built around a Raspberry Pi 4, which is placed as an inline bridge between the router and the internal SME network. This setup allows the Raspberry Pi to observe live traffic in real time while keeping the deployment simple and low-cost [3], [4].

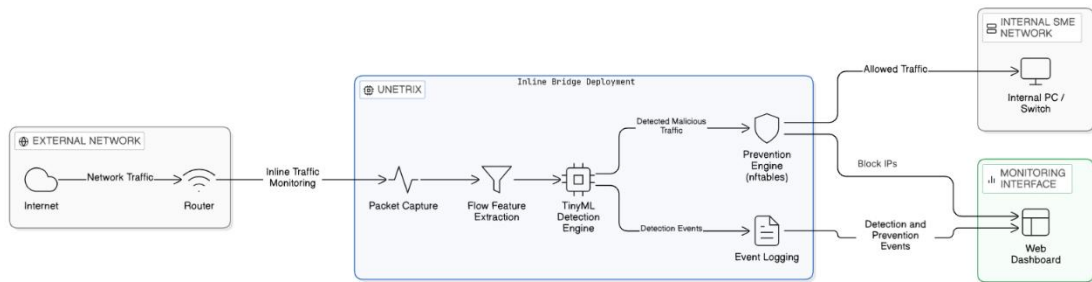


Figure 3.2: System Architecture Diagram

As shown in Figure 3.2, the architecture consists of four main parts: the external network, the Raspberry Pi-based security device, the internal SME network, and the web dashboard. Traffic first arrives from the router and passes through the Raspberry Pi. Inside Raspberry Pi, the system performs packet capture, flow feature extraction, attack detection, prevention handling, and event logging.

Benign traffic is forwarded to the internal network, while malicious traffic can trigger blocking through nftables.

The dashboard is connected to the Raspberry Pi to provide monitoring support. It displays detected attacks, blocked IP addresses, alert information, and other event details. Therefore, the proposed architecture combines real-time monitoring, attack detection, temporary prevention, and dashboard-based visibility in a single edge device, making it suitable for practical SME deployment [3], [5].

3.5 Inline Deployment and Runtime Workflow

The proposed system was deployed on a Raspberry Pi 4 as an inline bridge between the router and the internal SME network. This setup allows all traffic entering or leaving the protected network segment to pass through the Raspberry Pi before reaching the internal devices. Such an edge-based deployment is suitable for small networks because it enables traffic inspection close to the source while still using low-cost hardware [3], [4].

At runtime, the workflow starts with live packet capture. Network packets are captured using Dumpcap, and the captured traffic is stored as rolling PCAP files. These PCAP files are then processed using the Python implementation of CICFlowMeter, which converts raw packet traces into flow-based records. This step is important because the final model was trained using flow-level features rather than raw packets [8].

Once the flow has been generated, the features are extracted and shaped according to the input format of the model. Then the saved preprocessing pipeline is applied to ensure the data structure at runtime is consistent with the training data. The feature vector is then fed into the deployed TensorFlow Lite model for a multiclass prediction, which outputs the class and confidence.

With the predicted result, the system determines whether the traffic is malicious or benign. If the traffic is determined to be malicious with confidence higher than the threshold, the IP is temporarily blocked by nftables. If the traffic is

classified as benign, or the confidence is below the threshold, the traffic is permitted. In either case, the decision is also logged and displayed on the web dashboard. This process allows the system to integrate packet capture, local inference, automated response and dashboard monitoring in the deployment process [3], [4].

3.6 Detection, Prevention, and Dashboard Integration

The system was designed to not only detect malicious traffic, but also to automatically react on it and provide monitoring information. Once the TensorFlow Lite model has generated a prediction, the system verifies the class predicted and the confidence of the prediction. If the traffic is predicted as Benign, no prevention mechanism is triggered and the traffic passes through. If the traffic is classified as malicious and the confidence is greater than the threshold, the prevention mechanism is triggered. This is done to prevent the unnecessary prevention mechanism and to make the solution more feasible for real-life implementation [6], [3].

The prevention mechanism is implemented via nftables. When a malicious flow with high confidence is identified, the corresponding source IP address is added to a blocking set with a timeout. This enables Raspberry Pi to block the IP address for a specific amount of time, rather than indefinitely. For this work, the block time was set to 10 minutes. This approach enables the system to function as a lightweight intrusion detection and prevention system (IDPS) rather than a lightweight IDS that only generates alerts. The need to move beyond alert-only in lightweight IDS has also been discussed in previous studies [3], [5].

To improve usability, the system is integrated with a web dashboard. The dashboard displays detected attack types, source and destination addresses, confidence scores, severity levels, blocked IP addresses, and alert history. It also shows whether an event was only logged, actively blocked, or already under blocking. This makes the system easier to monitor in SME environments, where administrators may need a simple interface to understand current threats and system actions without reading low-level firewall rules or logs [6], [5].

4. IMPLEMENTATION

This chapter explains how the proposed system was implemented as a real-time network threat detection and prevention solution for SME environments. The implementation was carried out on a Raspberry Pi 4, which was used as the main edge device for traffic monitoring, local inference, automated prevention, and dashboard integration. The purpose of this chapter is to show how the design presented in Chapter 3 was converted into a working prototype that could operate on real network traffic.

The implementation combines several components into one complete pipeline. These include live packet capture, flow-based feature extraction, TensorFlow Lite model inference, temporary IP blocking using nftables, and a web dashboard for monitoring detected attacks and blocked IPs. Similar studies have also shown that Raspberry Pi can be used as a practical platform for lightweight intrusion detection in constrained or small-scale environments [3], [5]. In this project, the implementation was focused not only on detection accuracy, but also on making the overall system usable in a real deployment setting.

4.1 Hardware Environment

The hardware platform used for the implementation was a Raspberry Pi 4 with 8 GB RAM. This device was selected because it provides enough processing capability for lightweight edge-based inference while still being affordable and practical for SME deployment. The Raspberry Pi acted as the main processing unit of the system and was placed as an inline bridge between the router and the internal network. This allowed all traffic entering or leaving the protected network segment to pass through the device before reaching the internal PC or switch.

The physical setup consisted of three main parts: the external network side, the Raspberry Pi-based security device, and the internal SME network side. The router represented the external traffic source, while the internal side consisted of a PC or switch connected through the Raspberry Pi. In this setup, the Raspberry Pi continuously observed network traffic, processed captured data,

and enforced prevention actions when malicious activity was detected. This kind of low-cost edge deployment is suitable for SMEs because it does not require expensive dedicated security appliances and can be inserted into an existing network with minimal structural changes [3], [9].

The hardware environment was selected to support the full runtime workflow of the proposed system, including packet capture, feature extraction, model execution, temporary blocking, and dashboard communication. Since the project aimed to build a practical real-time solution rather than only an offline classifier, using Raspberry Pi as the deployment device was an important part of the implementation. Previous Raspberry Pi-based intrusion detection studies also support the use of this platform for lightweight network security applications [5], [9].

4.2 Software Environment

The proposed system was implemented using a combination of software tools that support real-time traffic monitoring, flow-based feature extraction, lightweight model inference, automated prevention, and dashboard-based monitoring. The main programming language used in the implementation was Python, since it provided a practical way to integrate traffic processing, machine learning inference, firewall control, and dashboard communication in a single runtime pipeline. Python is also commonly used in lightweight IDS research because it allows rapid integration of data processing and model deployment components [3], [4].

For live traffic capture, the system used Dumpcap, which continuously captured packets passing through the Raspberry Pi bridge and stored them as rolling PCAP files. These PCAP files were then processed using the Python implementation of CICFlowMeter to convert raw traffic into flow-based records. This was necessary because the final model was trained using flow-level features rather than raw packets. The use of CICFlowMeter-style flow generation is consistent with the original CICIDS2017 dataset design, where

network traffic was converted into flow-based features for intrusion detection research [8].

The machine learning part of the system used a fine-tuned MLP model deployed in TensorFlow Lite float32 format. At runtime, the system loaded the saved TFLite model, the scaler file, and the label encoder to process incoming flow features and generate class predictions. For prevention, nftables was used to apply temporary IP blocking when malicious traffic was detected with high confidence. In addition, a web dashboard was implemented to display detected attacks, blocked IPs, confidence scores, and alert logs. Together, these software components formed the full runtime environment of the proposed Raspberry Pi-based intrusion detection and prevention system.

4.3 Real-Time Packet Capture and Flow Conversion

The real-time operation of the proposed system begins with live packet capture on Raspberry Pi 4. Since the device is placed as an inline bridge between the router and the internal network, all passing traffic can be observed directly. For this purpose, Dumpcap was used as the packet capture tool. It continuously collects packets from the network interface and stores them as rolling PCAP files. Dumpcap was selected because it is lightweight and suitable for continuous packet capture in a real-time environment.

After packet capture, the generated PCAP files are passed to the Python implementation of CICFlowMeter for flow conversion. This step is necessary because the final machine learning model was trained using flow-based features, not raw packet data. The flow converter groups packets into bidirectional flows and calculates the required traffic statistics for each flow. This method is consistent with the feature-generation approach used in CICIDS2017, where network traffic was transformed into flow-level records for intrusion detection experiments [8].

Once the flow records are created, the system extracts only the selected features required by the deployed model. These features are then prepared for the inference stage by matching the expected input format of the trained model.

This flow-based approach is useful because it reduces the complexity of handling raw packet streams directly and provides structured behavioral information that can be used effectively for attack detection [8]. Therefore, the packet capture and flow conversion stage acts as the main link between live network traffic and the machine learning inference module of the proposed system.

4.4 Inference Module Implementation

The inference module was implemented to perform real-time traffic classification on the Raspberry Pi after flow conversion and feature preparation. Once the required flow features are extracted, the system selects the final 20 features used by the trained model and applies the saved preprocessing pipeline. In this implementation, the saved scaler is loaded first and used to transform the runtime feature values into the same format used during training. This step is important because the deployed model expects input data in the same scaled form as the training data.

After preprocessing, the transformed feature vector is passed to the deployed TensorFlow Lite float32 model. TensorFlow Lite was used because it provides a lightweight inference format that is more suitable for edge devices than a full training framework. The model then produces class probabilities for the six target classes: Benign, Botnet, Brute Force, DDoS, DoS, and Reconnaissance. The class with the highest probability is selected as the prediction result, and its probability value is used as the confidence score. Lightweight edge-based IDS studies have similarly used compact model deployment strategies to support practical inference on constrained hardware [3], [4].

The inference process was designed to run continuously as new flow records became available. Since the final system is intended for real-time operation, the inference module was kept as lightweight as possible. Runtime testing on Raspberry Pi showed that the deployed TensorFlow Lite model can perform prediction with very low inference latency, while the larger share of runtime cost comes from preprocessing rather than from the model itself. This confirms

that the chosen model and deployment method are suitable for edge-based network threat detection in SME environments [3], [6].

4.5 Prevention Module Implementation

The prevention module was implemented to automatically respond when the system detects high-confidence malicious traffic. After the inference module produces a predicted class and confidence score, the result is checked against the decision rule used in this study. If the predicted traffic is **Benign**, no blocking action is taken. If a **single flow** is classified as malicious with a confidence score greater than **95%**, the system immediately triggers the prevention step. This confidence-based policy was selected to reduce unnecessary blocking while still allowing the system to react quickly to highly suspicious traffic.

To apply the prevention action, the system uses **nftables** on the Raspberry Pi. When a high-confidence malicious flow is detected, the corresponding source IP address is inserted into a dedicated blocking set. Once the rule is added, **all subsequent traffic from that source IP address is dropped** by nftables until the blocking rule expires or is manually removed. In this implementation, the blocked IP address is not kept permanently. Instead, a **timeout-based rule** is used so that the IP address is automatically removed after the defined blocking duration. For this project, the timeout period was set to **10 minutes**. This allows the system to provide temporary automated prevention without permanently blocking a host based on a single detection event.

The prevention logic was also integrated with the monitoring component. Whenever a blocking action is triggered, the event is recorded with details such as the predicted attack type, source IP address, confidence score, timestamp, and applied action. If the same IP address is detected again while it is already present in the blocking set, the system identifies it as already blocked and records the event accordingly. This improves traceability and makes the prevention process easier to monitor through the dashboard.

Overall, the prevention module allows the proposed system to operate as a lightweight **intrusion detection and prevention system** rather than only a passive IDS. The current implementation uses **single-flow confidence-based blocking** with **timeout-based persistence** in nftables, which makes it suitable for practical real-time protection in SME environments.

4.6 Dashboard Implementation

A dashboard was developed to offer an easy and convenient monitoring solution for the proposed system. It is essentially used to display the action of the detection and prevention modules. Rather than the user having to look directly at the logs and/or the firewall rules, the dashboard provides the necessary information in a more readable way. This is particularly beneficial in SMEs where administrators may need to get a fast glance at attacks and actions without having to use advanced security software [3], [5].

The dashboard is fed information from the Raspberry Pi runtime pipeline for each new flow. The system logs details of the events, including the type of attack, source IP, destination IP, confidence, severity, timestamp and action. The action field indicates whether the attack was only logged, whether the IP was blocked or if the IP was already blocked. This allows the user to determine what was detected, and what action was taken.

In addition to the event log, the dashboard also includes a separate section for blocked IP addresses. This panel helps the user identify which sources are currently under temporary prevention. Summary indicators such as the total number of alerts, blocked IPs, and critical detections are also shown to give a quick overview of the current security state. By combining alert history, response status, and block information in one interface, the dashboard supports the practical usability of the whole system.

Overall, the dashboard acts as the monitoring layer of the proposed solution. It connects the detection and prevention modules with a user-friendly interface, allowing the system to function not only as a technical security prototype, but

also as a manageable and observable platform for SME network protection [3], [9].

5. RESULTS AND DISCUSSION

5.1 Dataset Results

After cleaning, filtering, and label harmonization, the final merged dataset contained 2,159,501 samples across six target classes: Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance. This dataset was created by combining selected files from CICIDS2017 and CSE-CIC-IDS2018, then removing unreliable or less suitable classes such as Infiltration, Web Attack, and Heartbleed. The final class structure was selected to better match the real deployment goal of the proposed system, which was to detect and prevent the main network threats relevant to SME environments [8], [7].

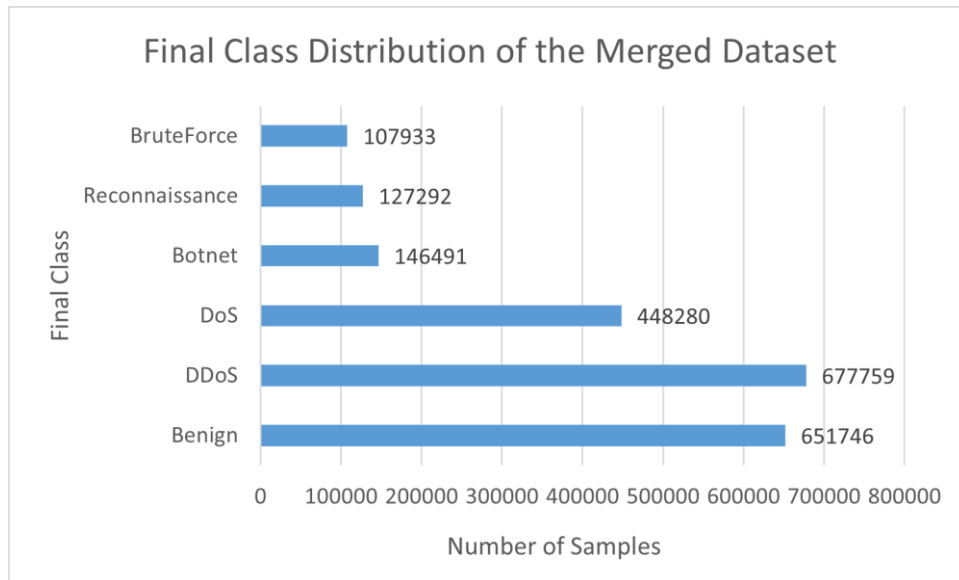


Figure 5.1: Final Class Distribution of the Merged Dataset

As shown in Figure 5.1, the largest class in the final dataset was DDoS with 677,759 samples (31.38%), followed by Benign with 651,746 samples (30.18%) and DoS with 448,280 samples (20.76%). The smaller classes were Botnet with 146,491 samples (6.78%), Reconnaissance with 127,292 samples (5.89%), and Brute Force with 107,933 samples (5.00%). This distribution

shows that the dataset was not perfectly balanced, but it still contained a strong representation of both benign and malicious traffic classes.

The final dataset distribution is important because it affects how the model learns different attack categories. The higher number of DDoS, Benign, and DoS samples gave the model more examples of these classes during training, while the smaller classes such as Brute Force and Reconnaissance had comparatively fewer samples. This type of class imbalance is common in intrusion detection research and must be considered when evaluating model performance, especially in multiclass classification tasks [7]. For this reason, later sections do not rely only on overall accuracy, but also examine per-class precision, recall, F1-score, and the confusion matrix.

Overall, the final dataset provided a large and practical training base for the proposed system. By reducing noisy classes and harmonizing the labels into six clear categories, the dataset became more suitable for training a lightweight multiclass intrusion detection model that could later be deployed on Raspberry Pi 4 for real-time operation.

5.2 Feature Selection Results

Feature selection was carried out to reduce the input space of the model and make the final system more suitable for Raspberry Pi deployment. Since the proposed solution was designed for real-time operation on a resource-constrained device, using all available traffic attributes was not practical. Previous lightweight intrusion detection studies have also shown that reducing the number of input features can help lower computational cost while still maintaining good detection performance [8], [3].

Before feature ranking, the non-predictive label-related columns were excluded from the merged dataset. A Random Forest feature importance method was then applied to rank the remaining flow-based features according to their contribution to the six-class classification task. Based on this ranking, the feature space was reduced to the top 20 selected features.

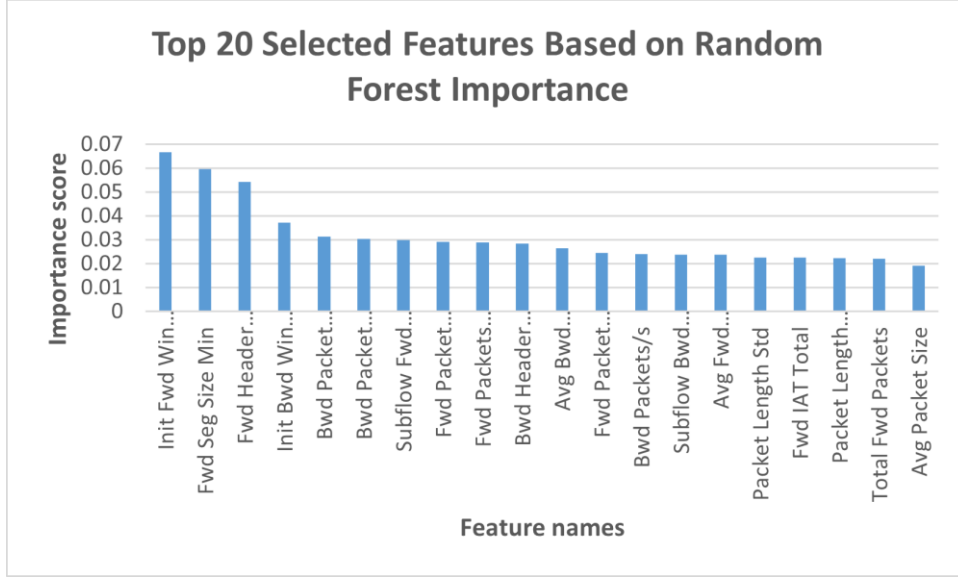


Figure 5.2: Top 20 Selected Features Based on Random Forest Importance

As shown in Figure 5.2, the most important features included Init Fwd Win Bytes, Fwd Seg Size Min, Fwd Header Length, Init Bwd Win Bytes, Bwd Packet Length Mean, and Bwd Packet Length Std. Other highly ranked features were related to sub flow bytes, packet-length statistics, packet rates, and flow timing behavior. These results suggest that the final model relied mainly on transport-level and flow-level behavioral characteristics rather than on a very large feature space.

The feature reduction stage was important for the final deployment. By reducing the input set to 20 key features, the preprocessing and inference pipeline became simpler and more efficient. This supported the development of a lightweight model that could still achieve strong multiclass detection performance while remaining suitable for edge execution on Raspberry Pi 4. Therefore, the feature selection results directly contributed to the practicality of the proposed real-time intrusion detection and prevention system [3], [8], [6].

5.3 Model Comparison Results

After feature selection, several baseline machine learning models were evaluated to identify the most suitable classifier for the proposed system. The evaluated models were Logistic Regression, Random Forest, XGBoost, and

Multilayer Perceptron (MLP). The purpose of this comparison was not only to find the most accurate model, but also to select a model that could be realistically deployed on Raspberry Pi 4 as part of a real-time detection and prevention pipeline. In lightweight intrusion detection research, model selection is usually influenced by both detection performance and deployment practicality, especially in constrained or edge-based environments where model size, latency, and resource usage are important design considerations [3], [4], [14].

Model	Accuracy	Macro F1	Model Size (MB)
Logistic Regression	0.77844	0.794104	0.001807
Random Forest	0.999451	0.999297	63.50695
XGBoost	0.9995	0.999399	2.819428
MLP	0.988662	0.990025	0.077419

Table 5.1: Model Comparison Results

The results showed clear differences between the tested models. **Logistic Regression** produced the weakest performance, with an accuracy of **77.84%** and a macro F1-score of **0.794**. In contrast, **Random Forest** and **XGBoost** achieved the strongest offline results, both reaching almost perfect classification performance. The **MLP** achieved a slightly lower but still very strong accuracy of **98.87%**, with a macro F1-score of **0.990**. Although Random Forest and XGBoost gave the highest classification scores, they were not automatically the best choices for deployment because model size and runtime suitability were also important considerations.

The final selected MLP was a **6-class multiclass classifier** with **20 input features** and **6 output neurons**. Based on hyperparameter tuning, the selected architecture consisted of **three hidden layers** with **32, 32, and 64 neurons**, respectively, using a **learning rate of 0.001**. The saved .keras model size was **79.28 KB**, and after conversion to **TensorFlow Lite float32** format, the

deployed model size was reduced to **18.96 KB**. This compact size made the model significantly more suitable for real-time Raspberry Pi deployment than larger alternatives such as **Random Forest (63.51 MB)** and **XGBoost (2.82 MB)**.

Therefore, the final model selection was based on a balance between **accuracy, compactness, and deployment practicality**, rather than accuracy alone. This type of trade-off is also emphasized in prior lightweight IDS studies, where reducing computational overhead while preserving strong detection performance is considered essential for practical edge deployment [3], [4], [14].

5.4 Final MLP Performance Evaluation

After comparing the baseline models, the MLP was selected as the final classifier for deployment. To evaluate its multiclass performance in more detail, the model was tested using the final six target classes: Benign, Botnet, Brute Force, DDoS, DoS, and Reconnaissance. The evaluation was based on overall accuracy, per-class precision, recall, F1-score, and the confusion matrix. These measures are important in intrusion detection because overall accuracy alone does not show how well the model performs for each attack category [3], [4].

Class	Precision	Recall	F1-score	Support
Benign	0.99	0.98	0.99	130349
Botnet	0.99	0.99	0.99	29298
BruteForce	1	0.99	0.99	21587
DDoS	0.99	1	0.99	135552
DoS	0.97	0.99	0.98	89656
Reconnaissance	1	1	1	25459
Accuracy			0.99	431901
Macro Average	0.99	0.99	0.99	431901
Weighted Average	0.99	0.99	0.99	431901

Table 5.2 Per-Class Performance of the MLP Model

The final MLP achieved an overall accuracy of 98.87%, with a macro F1-score close to 0.99. The per-class results showed strong performance across all six classes. Reconnaissance achieved the best performance, with precision, recall, and F1-score of 1.00. Benign, Botnet, Brute Force, and DDoS were also classified very accurately, each achieving values close to 0.99. Among the six classes, DoS showed the lowest F1-score, although it still performed strongly at around 0.98.

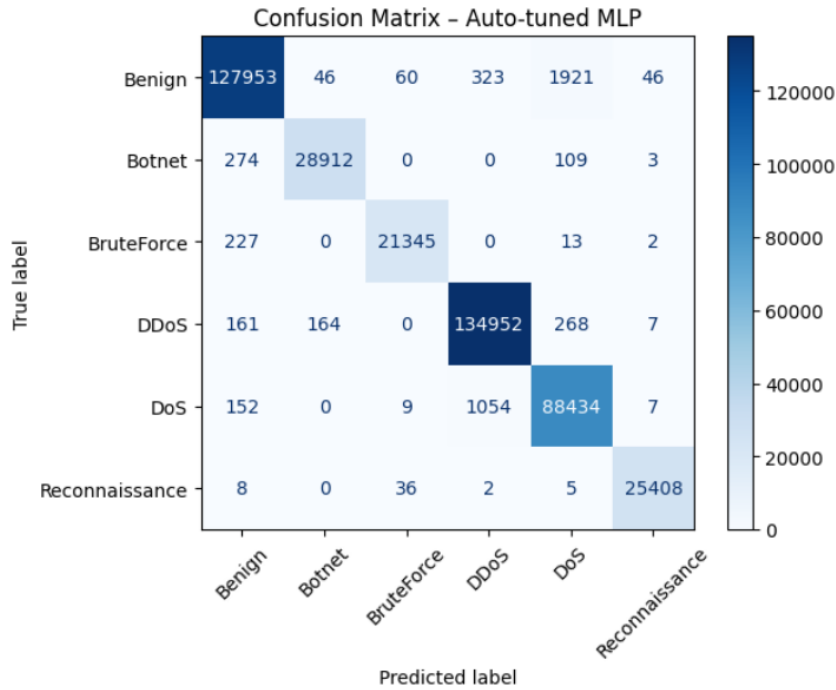


Figure 5.3: Confusion Matrix of MLP Model

The confusion matrix gives a clearer view of the misclassifications. Most predictions were concentrated along the main diagonal, which shows that the model learned to separate the classes effectively. The most noticeable confusion occurred between DoS and DDoS. In particular, a number of DoS samples were predicted as DDoS. This is understandable because both attacks can produce similar flow-level behavior such as high packet rates and abnormal traffic volume. A small number of benign flows were also misclassified as DoS or DDoS, which suggests that some normal traffic patterns may partially resemble attack behavior at the flow level.

Even with these minor misclassifications, the final MLP showed strong and stable multiclass performance. The results confirm that the selected 20-feature input set was sufficient to support accurate classification, while the model remained compact enough for deployment on Raspberry Pi. Therefore, the final MLP provided a good balance between classification quality and lightweight deployment suitability, which was the main objective of this project [3], [6].

5.5 Runtime Performance on Raspberry Pi

To evaluate whether the proposed system was practical for real-time use, the deployed model was tested on **Raspberry Pi 4** using runtime performance measurements. Since the final system was designed for edge deployment, it was important to measure not only classification accuracy but also how quickly the model could perform prediction in the actual hardware environment. Prior lightweight IDS studies have similarly emphasized that runtime efficiency, low computational cost, and suitability for constrained devices are key requirements when machine learning models are deployed on Raspberry Pi or other edge platforms [3], [4], [5].

Measurement Type	Samples Tested	Average Time (ms)	Minimum Time (ms)	Maximum Time (ms)
Pure Inference	1000	0.0202	0.0196	0.0442
Preprocessing + Inference	1000	3.8578	3.734	12.4236

Table 5.3 Runtime Performance on Raspberry Pi

Two timing experiments were carried out. The first measured **pure inference time**, where already scaled input samples were passed directly to the deployed TensorFlow Lite model. In this case, the model achieved an average inference time of **0.0202 ms**, with a minimum of **0.0196 ms** and a maximum of **0.0442 ms** over 1,000 samples. This result shows that the deployed TFLite model itself is extremely lightweight and suitable for fast edge-based prediction.

The second experiment measured **preprocessing plus inference time**, which is a more realistic runtime scenario because the feature values still needed to be transformed using the saved scaler before prediction. In this case, the system recorded an average time of **3.8578 ms**, with a minimum of **3.7340 ms** and a maximum of **12.4236 ms**. Compared to the pure inference result, this shows that the larger share of runtime cost comes from preprocessing rather than from the TensorFlow Lite model itself.

The lightweight deployment suitability of the selected model is also reflected in its final file size. The saved .keras model size was **79.28 KB**, and after conversion to **TensorFlow Lite float32** format, the deployed model size was reduced to **18.96 KB**. This compact TFLite size further supports the suitability of the selected MLP model for real-time execution on Raspberry Pi 4, where both memory footprint and inference efficiency are important practical considerations.

Overall, these results indicate that the deployed model is highly efficient on Raspberry Pi 4. The prediction stage itself introduces very little delay, while the additional runtime cost mainly comes from preprocessing. Even so, the total preprocessing plus inference time remained low enough to support real-time operation in the proposed intrusion detection and prevention pipeline. This is consistent with earlier lightweight IDS studies showing that resource-constrained edge devices can support practical real-time attack detection when compact models and efficient feature pipelines are used [3], [4], [5].

5.6 Resource Usage Analysis

In addition to inference speed, the resource usage of the deployed system was also measured on the Raspberry Pi 4. This was important because the proposed solution was intended for continuous real-time operation on a low-cost edge device. For this reason, CPU and memory usage were monitored while the packet capture, inference, and logging pipeline were active. Resource-aware evaluation is commonly discussed in lightweight IDS research because edge

devices must handle security tasks without consuming excessive hardware resources [3], [8].

Component	Avg CPU (%)	Max CPU (%)	Avg RAM (MB)	Max RAM (MB)
Raspberry Pi system overall	12.62	38.6	374.74	474.37
Python inference process	1.2	4	138.53	138.58
Dumpcap	0.03	1	8.43	8.43

Table 5.4: Resource Usage Analysis

The results showed that the overall system recorded an average CPU usage of 12.62%, with a peak of 38.60% during heavier processing periods. The average system memory usage was 374.74 MB, while the maximum observed memory usage reached 474.37 MB. These values indicate that the full run-time pipeline could operate on the Raspberry Pi without exhausting the available hardware resources.

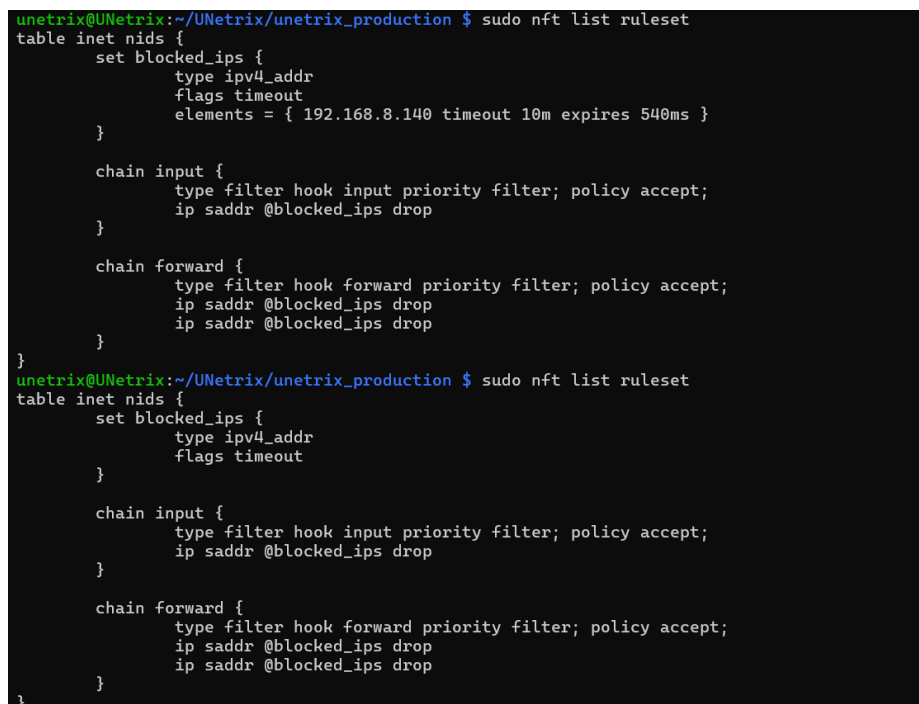
At the process level, the Python inference process used an average CPU of 1.20% and an average memory usage of 138.53 MB, with a maximum CPU of 4.00% and maximum memory usage of 138.58 MB. The Dumpcap packet-capture process showed very small overhead, with an average CPU usage of only 0.03% and stable memory usage of 8.43 MB. This suggests that the inference process itself remained lightweight, while the larger part of resource consumption came from the wider runtime environment rather than the deployed model alone.

Overall, the resource usage results support the practicality of the proposed system for SME deployment. The Raspberry Pi 4 was able to run the detection and prevention pipeline with moderate system resource consumption, while the machine learning inference component remained lightweight. These findings are consistent with previous studies that showed Raspberry Pi can be used as a

practical platform for lightweight intrusion detection and security monitoring in constrained environments [3], [4], [5].

5.7 Prevention Validation Using nftables

To verify that the proposed system could perform real prevention actions, the nftables-based blocking mechanism was tested on the Raspberry Pi deployment. The purpose of this test was to confirm that the system not only detects malicious traffic but also takes an automatic response by blocking suspicious source IP addresses when the confidence score is above the defined threshold. This is an important part of the project because many lightweight IDS solutions focus mainly on detection and alerting, while active prevention is often limited or not fully implemented [5], [6].



```
unetrix@UNetrix:~/UNetrix/unetrix_production $ sudo nft list ruleset
table inet nids {
    set blocked_ips {
        type ipv4_addr
        flags timeout
        elements = { 192.168.8.140 timeout 10m expires 540ms }
    }

    chain input {
        type filter hook input priority filter; policy accept;
        ip saddr @blocked_ips drop
    }

    chain forward {
        type filter hook forward priority filter; policy accept;
        ip saddr @blocked_ips drop
        ip saddr @blocked_ips drop
    }
}
unetrix@UNetrix:~/UNetrix/unetrix_production $ sudo nft list ruleset
table inet nids {
    set blocked_ips {
        type ipv4_addr
        flags timeout
    }

    chain input {
        type filter hook input priority filter; policy accept;
        ip saddr @blocked_ips drop
    }

    chain forward {
        type filter hook forward priority filter; policy accept;
        ip saddr @blocked_ips drop
        ip saddr @blocked_ips drop
    }
}
```

Figure 5.4: nftables Block Lifecycle

During validation, the ruleset inspection confirmed that the system created a dedicated blocked_ips set inside the nids table and inserted detected malicious IP addresses into that set. The blocking rule was applied to both the input and forward chains, which allowed the Raspberry Pi to drop malicious traffic at the inline bridge level. The screenshot evidence also showed that the blocked IP

entry was assigned a 10-minute timeout, which matched the prevention policy used in the implementation.

A second ruleset inspection was carried out after the timeout period expired. At this stage, the previously blocked IP address had been automatically removed from the `blocked_ips` set. This confirmed that the prevention mechanism supports temporary blocking rather than permanent firewall insertion. This is useful in practical deployment because it allows the system to react to high-confidence threats without keeping an address blocked indefinitely from a single detection result.

Overall, the nftables validation confirmed that the proposed system works as a lightweight intrusion detection and prevention system (IDPS), not just a passive IDS. The results showed that high-confidence malicious traffic could trigger automatic timeout-based blocking and that the block lifecycle was correctly enforced by the Raspberry Pi in real time. This makes the prevention component one of the key practical strengths of the proposed solution [3], [5], [6].

5.8 Dashboard Validation

The dashboard was evaluated to confirm that the proposed system can provide clear and useful monitoring support during real-time operation. In this project, the dashboard was not treated as only a visual add-on, but as an important part of the full detection and prevention workflow. Its role was to display attack events, show prevention actions, and help the user understand the current security state of the network. This is especially useful in SME environments, where administrators may need a simple and practical interface instead of manually checking raw logs or firewall rules [3], [5].

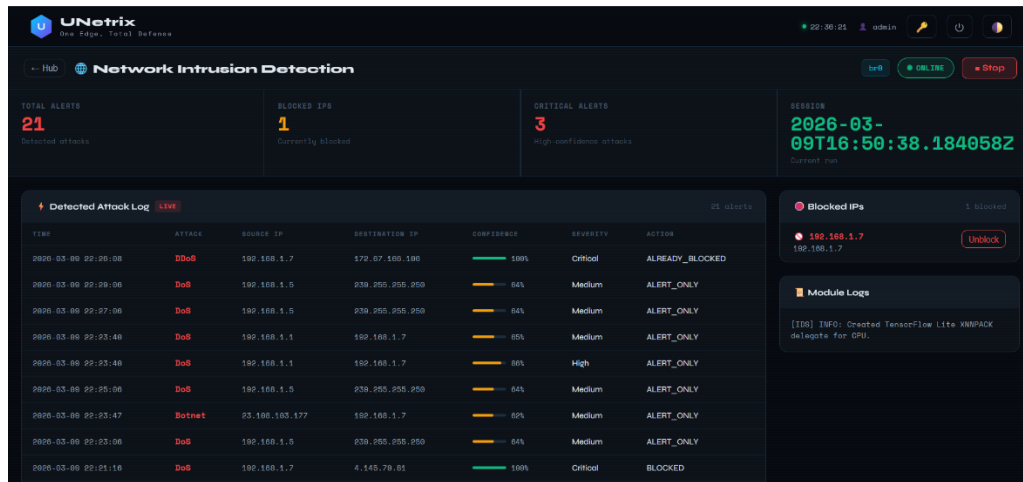


Figure 5.5: NIDS Web Dashboard

The dashboard validation showed that the system was able to record and display important event details in real time. These included the detected attack type, source and destination IP addresses, confidence score, severity level, timestamp, and response action. The dashboard also distinguished between different action states such as ALERT, BLOCKED, and ALREADY_BLOCKED, which made it easier to understand how the system responded to each event. In addition, the blocked IP section confirmed that prevention actions applied by nftables were also reflected in the monitoring interface.

The dashboard also provided summary indicators such as the total number of alerts, currently blocked IPs, and critical detections. These values gave a quick overview of the system status without requiring the user to inspect each event individually. This type of monitoring support improves the practical usability of the system because it allows users to observe both detection and prevention behavior from a single interface.

Overall, the dashboard validation confirmed that the proposed system provides not only real-time attack detection and temporary prevention, but also clear operational visibility. This supports the practical deployment goal of the project, since a security solution for SMEs should be understandable and manageable as well as technically effective [3], [9].

5.9 Discussion

The results of this study show that the proposed system was able to achieve a good balance between detection performance, lightweight deployment, and practical usability. The final MLP model achieved strong multiclass classification results, while the Raspberry Pi-based deployment showed that the system could operate with low inference time and moderate resource usage. This is important because the main goal of the project was not only to build an accurate classifier, but also to develop a solution that could work in a real SME environment using affordable hardware [3], [4].

One of the most important findings is that the final deployed model was not the most accurate offline model, but it was the most practical one. Random Forest and XGBoost produced slightly better offline results, but their model sizes made them less suitable for lightweight edge deployment. In contrast, the selected MLP remained highly accurate while being much smaller and easier to convert into TensorFlow Lite format. This shows that, in edge-based intrusion detection, model selection should not depend only on accuracy, but also on compactness and deployment efficiency [3], [6].

The confusion matrix also gave useful insight into the behavior of the final model. Most classes were classified correctly, but the main overlap appeared between DoS and DDoS. This is understandable because both attack types can create similar flow-level patterns, such as abnormal packet rates and high traffic volume. A small number of benign samples were also misclassified as attack traffic. This suggests that although the model performs strongly overall, some traffic behaviors still overlap when they are represented using flow-based features [4], [7].

Another important result was the runtime performance on Raspberry Pi 4. The measured pure inference time was extremely low, while the preprocessing plus inference time was noticeably higher. This shows that the main runtime overhead comes from preprocessing rather than from the TensorFlow Lite model itself. In other words, the model was lightweight enough for real-time

use, and future improvements could focus more on making the preprocessing stage faster and more efficient [3], [6].

The prevention and monitoring results further strengthen the practical value of the system. The nftables validation confirmed that the Raspberry Pi could automatically apply temporary blocking for malicious IP addresses and remove them again after the timeout period. The dashboard validation also showed that detected attacks, blocked IPs, confidence scores, and response actions could be viewed clearly in real time. Together, these results show that the proposed system is more than just a detection-only prototype, since it combines detection, automated response, and monitoring in one deployment pipeline [6], [5].

Overall, the discussion of the results shows that the proposed system was successful in delivering a practical and lightweight security solution for SMEs. Although some limitations remain, especially in class overlap and dataset dependence, the final implementation demonstrates that a Raspberry Pi-based TinyML approach can provide meaningful real-time network threat detection and prevention in a small-network environment.

5.10 Limitations

Although the proposed system achieved strong results, there are several limitations that should be considered. First, the model was developed and evaluated mainly using the public datasets CICIDS2017 and CSE-CIC-IDS2018. These datasets are widely used in intrusion detection research, but previous studies have reported issues such as labeling errors, duplicated records, and feature-generation problems [8], [7]. Because of this, the performance achieved in this study may not fully represent all real-world SME network conditions.

Another limitation is that the final model supports only six traffic classes: Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance. This means the current system cannot correctly identify attacks outside these selected categories. Some classes such as Infiltration, Web Attack, and Heartbleed were removed during preprocessing due to reliability issues or low sample counts, so

the final solution is focused on the most relevant attack classes rather than a fully general intrusion detection problem [8].

The results also showed that some overlap still exists between certain classes, especially DoS and DDoS. Since these attacks can generate similar flow-level patterns, the model may confuse them in some cases. In addition, a small number of benign flows were misclassified as attack traffic. This means that even though the overall performance is high, the possibility of false positives and false negatives still remains, which is an important practical issue in intrusion detection systems [3], [4].

From the deployment side, the runtime evaluation showed that the TensorFlow Lite model itself was very fast, but preprocessing contributed more to latency than the model inference stage. This means future improvements should focus not only on the classifier, but also on reducing preprocessing overhead. Another limitation is that the deployment testing was carried out on a Raspberry Pi prototype in a controlled environment. Larger and more diverse real-world SME networks may introduce additional traffic patterns, scalability challenges, and operational differences that were not fully covered in this work [3], [9].

Overall, these limitations do not reduce the value of the proposed system, but they show that the current implementation should be viewed as a practical lightweight prototype rather than a complete solution for every network environment. Future work can improve the system by adding more attack classes, reducing preprocessing overhead, and evaluating the deployment in more diverse live network settings.

6. COMMERCIALIZATION AND PRACTICAL DEPLOYMENT

This chapter discusses the practical and commercial relevance of the proposed system in the context of small and medium-sized enterprises (SMEs). While the previous chapters focused on the technical design, implementation, and evaluation of the system, this chapter considers how the solution can be viewed as a low-cost and usable security product for real-world deployment. In SME environments, security solutions must not only be effective, but also affordable,

lightweight, and easy to manage. This makes commercialization an important part of the study, especially because the proposed system was designed around a low-cost Raspberry Pi 4 platform and a lightweight deployment approach [1], [9].

The proposed system combines real-time network threat detection, temporary automated prevention, and dashboard-based monitoring in a single edge device. From a practical point of view, this creates value because SMEs often need a security solution that can provide useful protection without requiring expensive enterprise hardware or advanced security expertise. Therefore, this chapter evaluates the proposed system not only as a technical prototype, but also as a solution with potential for practical SME adoption and future product development [1], [9].

6.1 SME Need and Practical Relevance

SMEs are increasingly dependent on digital systems, online services, and connected networks for their daily operations. At the same time, they are becoming more exposed to cyber threats, while often having fewer resources to defend themselves compared to large organizations. Research has shown that SMEs experience real financial and operational risks from cybercrime, but many still lack strong technical protection, dedicated security staff, or the budget needed for advanced enterprise-level security solutions [1]. This creates a clear need for security systems that are both effective and affordable.

The proposed system is relevant in this context because it was designed as a low-cost edge-based security solution. It uses Raspberry Pi 4 as the main deployment platform, which makes the hardware requirement much lighter than traditional enterprise appliances. Previous studies have also suggested that Raspberry Pi can be a practical platform for cyber defense in SME and smart-home environments, especially when cost, flexibility, and deployment simplicity are important factors [9]. In addition, lightweight IDS research shows that edge-based monitoring can be useful in constrained environments where centralized or resource-heavy approaches may not be practical [3], [4].

Another reason for the practical relevance of the proposed system is that it does more than just detect attacks. The system also provides temporary automated blocking using nftables and a dashboard for real-time monitoring. This is important for SMEs because many smaller organizations need security solutions that are easy to understand and manage, rather than systems that only generate complex logs or alerts. By combining detection, prevention, and visibility in one platform, the proposed system offers a more practical form of network protection that is aligned with the needs of small-scale environments.

Overall, the proposed system is practically relevant because it addresses a common SME problem: the need for a security solution that is affordable, lightweight, easy to deploy, and capable of providing real-time protection. This makes it a suitable candidate for further development as a deployable product or service for small business environments [1], [9].

6.2 Cost and Deployment Considerations

One of the main strengths of the proposed system is its low-cost deployment model. Unlike traditional enterprise network security appliances, the proposed solution uses a Raspberry Pi 4 as the main edge device, which significantly reduces the hardware cost of implementation. This makes the system more attractive for SMEs, because many smaller organizations cannot afford expensive dedicated intrusion detection and prevention appliances. Previous studies have also highlighted Raspberry Pi as a practical platform for low-cost cyber defense in constrained or small-network environments [3], [9].

From a deployment perspective, the system is also relatively simple to place within an SME network. Since the Raspberry Pi is used as an inline bridge between the router and the internal network, the solution can be inserted into an existing network without requiring major infrastructure changes. This is useful in practice because SMEs often need security improvements that do not involve complex installation or large-scale redesign of their current network setup. Earlier Raspberry Pi-based security studies have also shown that edge and

gateway-style deployment is practical when compact and low-cost devices are used for monitoring and protection [3], [5].

Another important consideration is the operational cost of the system. The runtime evaluation showed that the deployed model has low inference time and moderate CPU and memory usage, which means that the Raspberry Pi can run the solution continuously without requiring high-performance computing resources. This supports the practicality of the system for long-term SME use, since lower processing demand generally means lower energy usage and easier maintenance compared to heavier server-based solutions. In addition, the use of open and widely available software components such as Python, TensorFlow Lite, Dumpcap, nftables, and a simple web dashboard further reduces the overall software cost of deployment.

However, some practical deployment considerations still remain. The system currently depends on proper inline placement, correct network configuration, and a controlled runtime workflow for packet capture and flow generation. In a larger or more complex SME environment, deployment may require extra network planning and performance tuning. Even so, the proposed system still offers a strong cost advantage compared to enterprise-grade appliances, especially for small offices or organizations that need a practical and affordable first layer of network threat detection and prevention [1], [9].

6.3 Potential Productization

The proposed system also shows potential to be developed beyond an academic prototype into a practical security product for SMEs. One possible product form is a small-edged security appliance that can be installed between the router and the internal network. In this form, the Raspberry Pi-based device would provide real-time threat detection, temporary automated blocking, and dashboard-based monitoring without requiring expensive enterprise hardware. This type of product would be especially useful for SMEs because many small businesses need affordable protection that is easy to deploy and manage [1], [9].

A second possibility is to develop the system as a managed security solution. In this model, the device could still run locally at the SME site, but monitoring, updates, and support could be handled by a service provider. This would be useful for small businesses that do not have dedicated cybersecurity staff. Since the proposed system already includes a dashboard and automated response mechanism, it can be viewed as a foundation for a simple managed detection and response service aimed at smaller organizations [1], [3].

The productization potential is also supported by the design choices of the system. The use of a lightweight model, TensorFlow Lite deployment, low-cost hardware, and open software tools helps reduce the overall cost of development and deployment. In addition, the integration of detection, prevention, and monitoring into one platform gives the system more practical value than a detection-only prototype. Previous lightweight IDS studies have also shown that low-overhead edge security systems can be realistic for constrained environments, which supports the possibility of turning this type of solution into a usable product [3], [5].

However, to move toward full productization, several improvements would still be needed. These include easier installation, better support for different network environments, automated update mechanisms, and broader attack coverage. Even with these limitations, the proposed system already demonstrates the core features needed for a practical SME-oriented product: low-cost deployment, real-time detection, automated temporary prevention, and a user-friendly monitoring interface. Therefore, the study shows clear potential for future development into a deployable security appliance or service for small business environments.

6.4 Future Scalability and Real-World Use

Although the proposed system was developed and tested as a Raspberry Pi-based prototype, it has potential to be extended for broader real-world use. In its current form, the system is suitable for small office or SME environments where network traffic volume is moderate, and a low-cost inline security device

can be placed between the router and the internal network. This makes it practical for basic real-time monitoring, multiclass threat detection, temporary prevention, and dashboard-based visibility without requiring expensive enterprise infrastructure [1], [9].

For future scalability, the system could be improved in several ways. One direction is to support a larger number of attack classes and more diverse traffic patterns, so that the solution can handle broader real-world network conditions. Another direction is to optimize the preprocessing stage, since the runtime evaluation showed that preprocessing contributes more to latency than the TensorFlow Lite model itself. Improving this stage would make the system more efficient for higher traffic loads and more complex deployment scenarios [3], [4].

The system could also be adapted for deployment in multiple branch offices or distributed SME environments. In such a case, several Raspberry Pi-based nodes could be placed at different network points, while a central dashboard or monitoring service could collect alerts and status information from each device. This would make the solution more useful for organizations with more than one office location. In addition, future versions could include remote configuration support, model update mechanisms, and more advanced reporting features to improve long-term usability.

From a real-world perspective, the proposed system already demonstrates that a lightweight edge device can provide meaningful network protection when it combines detection, prevention, and monitoring in one platform. With further improvement in scalability, attack coverage, and deployment management, the system could become more practical for wider SME adoption and possibly for other constrained environments such as branch offices, smart buildings, or small industrial networks [3], [9].

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

This study presented a TinyML-powered real-time network threat detection and prevention system for SMEs using Raspberry Pi 4 as a lightweight edge device. The main aim of the project was to develop a practical and low-cost solution that could detect multiple network threats in real time, apply automated temporary blocking, and provide monitoring support through a dashboard. Based on the overall results, the proposed system was able to achieve this aim successfully by combining dataset preparation, feature selection, lightweight model deployment, automated prevention, and real-time monitoring in a single platform [2]- [4].

A cleaned and harmonized multiclass dataset was first prepared using selected files from CICIDS2017 and CSE-CIC-IDS2018. After preprocessing, the final dataset was reduced to six target classes: Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance. Feature selection was then applied to reduce the input space to the top 20 most important features, making the final model more suitable for lightweight deployment. Several baseline models were compared, and although Random Forest and XGBoost produced slightly higher offline accuracy, the MLP was selected as the final model because it provided the best balance between strong performance and compact deployment size [2], [3], [8].

The final model was deployed in TensorFlow Lite float32 format on Raspberry Pi 4 and integrated into a real-time pipeline that included packet capture, flow conversion, preprocessing, inference, prevention, and dashboard logging. The runtime evaluation showed that the deployed model achieved very low inference time and moderate resource usage, confirming that the proposed solution is suitable for edge execution. The nftables-based prevention mechanism also worked successfully by temporarily blocking high-confidence malicious IP addresses and removing them automatically after the timeout period. In addition, the dashboard validation confirmed that the system could

provide useful real-time visibility into detected attacks, blocked IPs, and response actions [3], [4], [5].

Overall, the study demonstrates that a Raspberry Pi-based TinyML approach can provide meaningful real-time network threat detection and prevention for SME environments. While the system still has limitations, such as dependence on benchmark datasets and restricted attack coverage, it shows that lightweight edge-based security solutions can be both practical and effective. Therefore, the proposed work contributes a realistic foundation for future SME-oriented network security systems that combine detection, automated response, and monitoring in a low-cost deployment model [2]- [5].

7.2 Future Work

Although the proposed system achieved strong results, there are several directions for future improvement. One important area is to expand the system beyond the current six-class setting and improve its ability to handle previously unseen or zero-day attacks. Recent research has shown growing interest in zero-day intrusion detection for IoT and constrained environments, since real-world attackers often use behaviors that are not fully represented in benchmark datasets. Because of this, future work can explore more adaptive learning methods, anomaly-based detection, or hybrid approaches that improve the system's ability to recognize unknown attack patterns rather than only the attacks included in the current training set [6], [17].

Another valuable improvement is the integration of Explainable Artificial Intelligence (XAI) into the detection pipeline. At present, the system shows the predicted attack class, confidence score, and prevention action through the dashboard, but it does not explain why a flow was classified as malicious. Recent IDS research has highlighted that explainability is important for improving trust, transparency, and usability, especially when machine learning models are used in security decisions [18]. In future work, XAI techniques such as SHAP or LIME could be added to show which flow features contributed most

to a prediction. This would make the dashboard more informative and help administrators better understand alerts and blocking decisions [18], [19].

A further improvement would be to optimize the preprocessing stage of the runtime pipeline. The deployment results showed that the TensorFlow Lite model itself was very fast, while preprocessing contributed more to total runtime latency. Therefore, future work can focus on reducing the cost of feature preparation, scaling, and data handling so that the system can process higher traffic volumes more efficiently in real deployments. This would improve scalability and make the solution more practical for larger SME environments or multiple branch offices [3], [4].

Future work can also test the system in more diverse live environments. The current implementation was validated in a controlled deployment setting, which was suitable for demonstrating the concept. However, real SME networks may contain different traffic patterns, device types, and operational conditions. Broader testing would help evaluate long-term stability, false positives, scalability, and the generalizability of the model outside benchmark dataset conditions [7], [9].

Overall, future work can build on the current system by improving zero-day attack detection, explainability, preprocessing efficiency, and real-world deployment testing. These directions would help move the proposed system from a practical prototype toward a more complete and trustworthy lightweight security platform for SME environments.

REFERENCES

- [1] M. F. Arroyabe, C. F. Arranz, F. D. Arroyabe and J. C. F. Arroyabe, "Revealing the realities of cybercrime in small and medium enterprises: Understanding fear and taxonomic perspectives," *Computers & Security*, vol. 141, 2024.
- [2] D. Pinto, I. Amorim, E. Maia and I. Praça, "A review on intrusion detection datasets, tools, processes, and features," *Computer Networks*, vol. 262, 2025.
- [3] X. H. Nguyen, X. D. Nguyen, H. H. Huynh and K. H. Le, "Realguard: A Lightweight Network Intrusion Detection System for IoT Gateways," *Sensors*, vol. 22, no. 2, 2022.
- [4] Y. N. Soe, Y. Feng, P. I. Santosa, R. Hartanto and K. Sakurai, "Towards a Lightweight Detection System for Cyber Attacks in the IoT Environment Using Corresponding Features," *Electronics*, vol. 9, no. 1, 2020.
- [5] V. Visoottiviseth, G. Chutaporn, S. Kungvanruttana and J. Paisarnduangjan, "PITI: Protecting Internet of Things via Intrusion Detection System on Raspberry Pi," in *2020 International Conference on Information and Communication Technology Convergence (ICTC)*, Jeju, 2020.
- [6] Z. Alwaisi, T. Kumar, E. Harjula and S. Soderi, "Securing constrained IoT systems: A lightweight machine learning approach for anomaly detection and prevention," *Internet of Things*, vol. 28, 2024.
- [7] L. Liu, G. Engelen, T. Lynar, D. Essam and W. Joosen, "Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *2022 IEEE Conference on Communications and Network Security (CNS)*, Austin, 2022.
- [8] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, Funchal, 2018.
- [9] S. S. Tirumala, N. Nepal and S. K. Ray, "Raspberry Pi-based Intelligent Cyber Defense Systems for SMEs and Smart-homes: An Exploratory Study," *EAI Endorsed Transactions on Smart Cities*, vol. 6, no. 18, 2022.

- [10] Z. Li and W. Yao, "A two stage lightweight approach for intrusion detection in Internet of Things," *Expert Systems with Applications*, vol. 257, 2024.
- [11] NIST, "nist.gov," 20 02 2007. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/94/final?utm_source=chatgpt.com. [Accessed 20 04 2026].
- [12] X. Chen, M. Liu, Z. Wang and Y. Wang, "Explainable Deep Learning-Based Feature Selection and Intrusion Detection Method on the Internet of Things," *Sensors*, vol. 24, no. 16, 2024.
- [13] A. Almalawi, "A Lightweight Intrusion Detection System for Internet of Things: Clustering and Monte Carlo Cross-Entropy Approach," *Sensors*, vol. 25, no. 7, 2025.
- [14] H. G. A. Umar, I. Yasmeen, M. Aoun, T. Mazhar, M. A. Khan, I. H. Jaghdam and H. Hamam, "Energy-efficient deep learning-based intrusion detection system for edge computing: a novel DNN-KDQ model," *Journal of Cloud Computing*, vol. 14, no. 32, 2025.
- [15] J. Huckelberry, Y. Zhang, A. Sansone, J. Mickens, P. A. Beerel and V. J. Reddi, "TinyML Security: Exploring Vulnerabilities in Resource-Constrained Machine Learning Systems," *Journal of Cloud Computing*, vol. 14, no. 32, 2025.
- [16] R. Custance and G. Mogos, "Raspberry pi Firewall and Intrusion Detection System," *INTERNATIONAL JOURNAL OF DESIGN, ANALYSIS AND TOOLS FOR INTERGRATED CIRCUITS AND SYSTEMS*, vol. 10, no. 1, 2021.
- [17] D. Krishnan, S. Singh and V. Sugumaran, "Explainable AI for zero-day attack detection in IoT networks using attention fusion model," *Discover Internet of Things*, vol. 5, no. 83, 2025.
- [18] O. Arreche, T. Guntur and M. Abdallah, "XAI-IDS: Toward Proposing an Explainable Artificial Intelligence Framework for Enhancing Network Intrusion Detection Systems," *Applied Sciences*, vol. 14, no. 10, 2024.
- [19] A. Alabbadi and F. Bajaber, "An Intrusion Detection System over the IoT Data Streams Using eXplainable Artificial Intelligence (XAI)," *Sensors*, vol. 25, no. 3, 2025.

APPENDICES

Appendix A: Dataset File Selection and Label Harmonization Details

A.1 Dataset Source

The datasets used in this study were **CICIDS2017** and **CSE-CIC-IDS2018**, which are widely used benchmark datasets in intrusion detection research because they contain both benign traffic and multiple modern attack categories. In this study, the datasets were accessed through publicly available **Kaggle mirrors** for download convenience, while the datasets are academically cited using their original published sources and review studies.

A.2 Original Dataset Labels

Original CICIDS2017 Labels:

- BENIGN
- DDoS
- PortScan
- Bot
- Infiltration
- Web Attack – Brute Force
- Web Attack – XSS
- Web Attack – Sql Injection
- FTP-Patator
- SSH-Patator
- DoS Hulk
- DoS GoldenEye
- DoS slowloris
- DoS Slowhttptest
- Heartbleed

Original CSE-CIC-IDS2018 Labels:

- Benign

- Bot
- SSH-Bruteforce
- FTP-BruteForce
- DDoS attacks-LOIC-HTTP
- DDOS attack-HOIC
- DDOS attack-LOIC-UDP
- DoS attacks-GoldenEye
- DoS attacks-Slowloris
- DoS attacks-Hulk
- DoS attacks-SlowHTTPTest
- Infiltration
- Brute Force -Web
- Brute Force -XSS
- SQL Injection

A.3 Label Harmonization

Since the two datasets used different naming conventions and contained overlapping attack categories, a label harmonization process was carried out before model training. Similar attack labels from both datasets were grouped into a smaller set of common final classes to create a more consistent and deployment-oriented multiclass classification problem. This type of preprocessing is important because previous studies have reported label inconsistencies, duplicated records, and reliability issues in CICIDS2017 and CSE-CIC-IDS2018.

Original Label	Final Class
BENIGN / Benign	Benign
DoS Hulk	DoS
DoS GoldenEye	DoS
DoS slowloris	DoS
DoS Slowhttpstest	DoS

DoS attacks-GoldenEye	DoS
DoS attacks-Slowloris	DoS
DoS attacks-Hulk	DoS
DoS attacks-SlowHTTPTest	DoS
DDoS	DDoS
DDoS attacks-LOIC-HTTP	DDoS
DDOS attack-HOIC	DDoS
DDOS attack-LOIC-UDP	DDoS
FTP-Patator	Brute Force
SSH-Patator	Brute Force
FTP-BruteForce	Brute Force
SSH-Bruteforce	Brute Force
Bot	Botnet
PortScan	Reconnaissance

A.4 Excluded Classes

Some original labels were excluded from the final dataset because of low sample counts or dataset-quality concerns. The excluded classes were:

- Infiltration / Infiltration
- Web Attack – Brute Force
- Web Attack – XSS
- Web Attack – Sql Injection
- Brute Force -Web
- Brute Force -XSS
- SQL Injection
- Heartbleed

Infiltration was removed because prior work reported reliability and corruption issues related to this class. **Web attack classes** and **Heartbleed** were excluded

because they had relatively low sample counts compared to the selected major traffic classes, making them less suitable for the final lightweight multiclass deployment setting [7].

Appendix B: Full Feature List and Selected Features

B.1 Original Feature Set

Before feature selection, the merged dataset contained **81 columns**, including traffic features and label-related fields. The original feature list is given below.

- | | |
|--------------------------------|----------------------------|
| 1. Destination Port | 24. Fwd IAT Max |
| 2. Flow Duration | 25. Fwd IAT Min |
| 3. Total Fwd Packets | 26. Bwd IAT Total |
| 4. Total Backward Packets | 27. Bwd IAT Mean |
| 5. Total Length of Fwd Packets | 28. Bwd IAT Std |
| 6. Total Length of Bwd Packets | 29. Bwd IAT Max |
| 7. Fwd Packet Length Max | 30. Bwd IAT Min |
| 8. Fwd Packet Length Min | 31. Fwd PSH Flags |
| 9. Fwd Packet Length Mean | 32. Bwd PSH Flags |
| 10. Fwd Packet Length Std | 33. Fwd URG Flags |
| 11. Bwd Packet Length Max | 34. Bwd URG Flags |
| 12. Bwd Packet Length Min | 35. Fwd Header Length |
| 13. Bwd Packet Length Mean | 36. Bwd Header Length |
| 14. Bwd Packet Length Std | 37. Fwd Packets/s |
| 15. Flow Bytes/s | 38. Bwd Packets/s |
| 16. Flow Packets/s | 39. Min Packet Length |
| 17. Flow IAT Mean | 40. Max Packet Length |
| 18. Flow IAT Std | 41. Packet Length Mean |
| 19. Flow IAT Max | 42. Packet Length Std |
| 20. Flow IAT Min | 43. Packet Length Variance |
| 21. Fwd IAT Total | 44. FIN Flag Count |
| 22. Fwd IAT Mean | 45. SYN Flag Count |
| 23. Fwd IAT Std | 46. RST Flag Count |

- | | |
|--------------------------|-----------------------------|
| 47. PSH Flag Count | 65. Subflow Bwd Packets |
| 48. ACK Flag Count | 66. Subflow Bwd Bytes |
| 49. URG Flag Count | 67. Init_Win_bytes_forward |
| 50. CWE Flag Count | 68. Init_Win_bytes_backward |
| 51. ECE Flag Count | 69. act_data_pkt_fwd |
| 52. Down/Up Ratio | 70. min_seg_size_forward |
| 53. Average Packet Size | 71. Active Mean |
| 54. Avg Fwd Segment Size | 72. Active Std |
| 55. Avg Bwd Segment Size | 73. Active Max |
| 56. Fwd Header Length.1 | 74. Active Min |
| 57. Fwd Avg Bytes/Bulk | 75. Idle Mean |
| 58. Fwd Avg Packets/Bulk | 76. Idle Std |
| 59. Fwd Avg Bulk Rate | 77. Idle Max |
| 60. Bwd Avg Bytes/Bulk | 78. Idle Min |
| 61. Bwd Avg Packets/Bulk | 79. label |
| 62. Bwd Avg Bulk Rate | 80. attack_family |
| 63. Subflow Fwd Packets | 81. attack_type |
| 64. Subflow Fwd Bytes | |

B.2 Final Selected Features

After applying Random Forest feature importance, the feature space was reduced to the top 20 selected features. These features were used as the final input set for model training, fine-tuning, and deployment.

1. Init Fwd Win Bytes
2. Fwd Seg Size Min
3. Fwd Header Length
4. Init Bwd Win Bytes
5. Bwd Packet Length Mean
6. Bwd Packet Length Std
7. Subflow Fwd Bytes
8. Fwd Packet Length Max
9. Fwd Packets Length Total

10. Bwd Header Length
11. Avg Bwd Segment Size
12. Fwd Packet Length Mean
13. Bwd Packets/s
14. Subflow Bwd Bytes
15. Avg Fwd Segment Size
16. Packet Length Std
17. Fwd IAT Total
18. Packet Length Max
19. Total Fwd Packets
20. Avg Packet Size

These selected features mainly represent packet length behavior, header information, subflow statistics, packet-rate characteristics, and flow timing information. Reducing the original feature space to these 20 features helped make the final model more suitable for lightweight real-time deployment on Raspberry Pi 4.

Appendix C: Final MLP Architecture and Hyperparameter Settings

The final deployed classifier used in this study was a **Multilayer Perceptron (MLP)** designed for **6-class multiclass classification**. The model used **20 selected flow-based input features** and predicted the following classes: **Benign, DoS, DDoS, Brute Force, Botnet, and Reconnaissance**.

Based on hyperparameter tuning, the final MLP architecture consisted of **three hidden layers** with **32, 32, and 64 neurons**, respectively. The selected **learning rate** was **0.001**. The main hyperparameter settings of the final model are shown in Table C.1

Hyperparameter	Value
Input features	20
Output classes	6
Hidden layers	3
Units in hidden layer 1	32
Units in hidden layer 2	32
Units in hidden layer 3	64
Learning rate	0.001
Batch size	128
Fine-tuning epochs	50

The compact size of the final model was one of the main reasons it was selected for deployment. The saved .keras model size was **79.28 KB**, and after conversion to **TensorFlow Lite float32** format, the deployed model size was reduced to **18.96 KB**.

Overall, the final MLP configuration supported the goal of developing a lightweight and practical model for real-time network threat detection on **Raspberry Pi 4**.